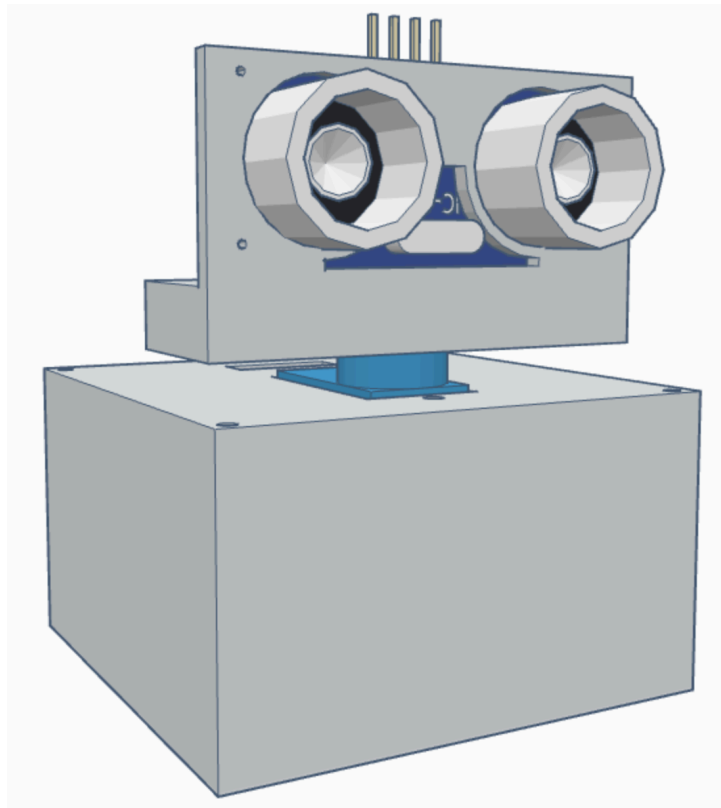


# **RADAR**

Progettazione di sistemi digitali



Alessio Manuelli - Samuele Bartorelli



**UNIVERSITÀ  
DI PISA**

# Indice

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>1. Introduzione.....</b>                                       | <b>2</b>  |
| <b>2. Architetture.....</b>                                       | <b>3</b>  |
| 2.1 Interfaccia Hardware.....                                     | 3         |
| 2.2 Strategie di Progetto.....                                    | 4         |
| <b>3. Codice Verilog - Descrizione dei Moduli Utilizzati.....</b> | <b>6</b>  |
| 3.1 misura_distanza.....                                          | 6         |
| 3.2 gestione_misura.....                                          | 8         |
| 3.3 gestione_display_7seg.....                                    | 9         |
| 3.4 servo_PWM.....                                                | 9         |
| 3.5 radar_VGA.....                                                | 11        |
| 3.6 VGA_sync.....                                                 | 13        |
| 3.7 sin_cos_ROM.....                                              | 15        |
| 3.8 clk_div.....                                                  | 16        |
| 3.9 reset_sincrono.....                                           | 16        |
| <b>4. Testbench.....</b>                                          | <b>17</b> |
| 4.1 Test di misura_distanza.....                                  | 17        |
| 4.2 Test di gestione_misura.....                                  | 18        |
| 4.3 Test di servo_PWM.....                                        | 19        |
| 4.4 Test di VGA_sync.....                                         | 19        |
| 4.4 Test di RADAR_VGA.....                                        | 22        |
| 4.5 Test di clk_div.....                                          | 24        |
| 4.6 Test del Reset.....                                           | 24        |
| <b>5. Analisi e Sintesi.....</b>                                  | <b>26</b> |
| 5.1 RTL Viewer.....                                               | 26        |
| 5.2 Summary Report.....                                           | 27        |
| 5.3 Resource Usage Report.....                                    | 27        |
| <b>6. Fitting - Place and Route.....</b>                          | <b>29</b> |
| 6.1 Fitter Summary.....                                           | 29        |
| 6.2 Resource Usage Summary.....                                   | 30        |
| 6.3 Layout.....                                                   | 31        |
| 7.1 Constraints di Clock.....                                     | 32        |
| 7.4 Summary of Paths.....                                         | 33        |
| 7.5 Data Arrival Path e Data required path.....                   | 34        |
| 7.6 Path in technology map viewer.....                            | 35        |
| 7.7 Path in chip planner.....                                     | 36        |
| <b>8. Conclusioni.....</b>                                        | <b>38</b> |

# **1. Introduzione**

Il progetto consiste nella realizzazione e nell'implementazione su FPGA di un radar, utilizzando il linguaggio Verilog per la descrizione della logica, e Quartus come ambiente di sviluppo. È stata utilizzata una scheda DE10 - Lite, la quale sfrutta una FPGA della serie MAX 10.

Viene mostrata l'architettura del sistema digitale realizzato, discutendo le strategie di progetto impiegate e la gestione di alcune criticità. Viene fornita una descrizione dei vari moduli Verilog utilizzati all'interno del progetto e vengono mostrati dei testbench esaustivi per ognuno di essi.

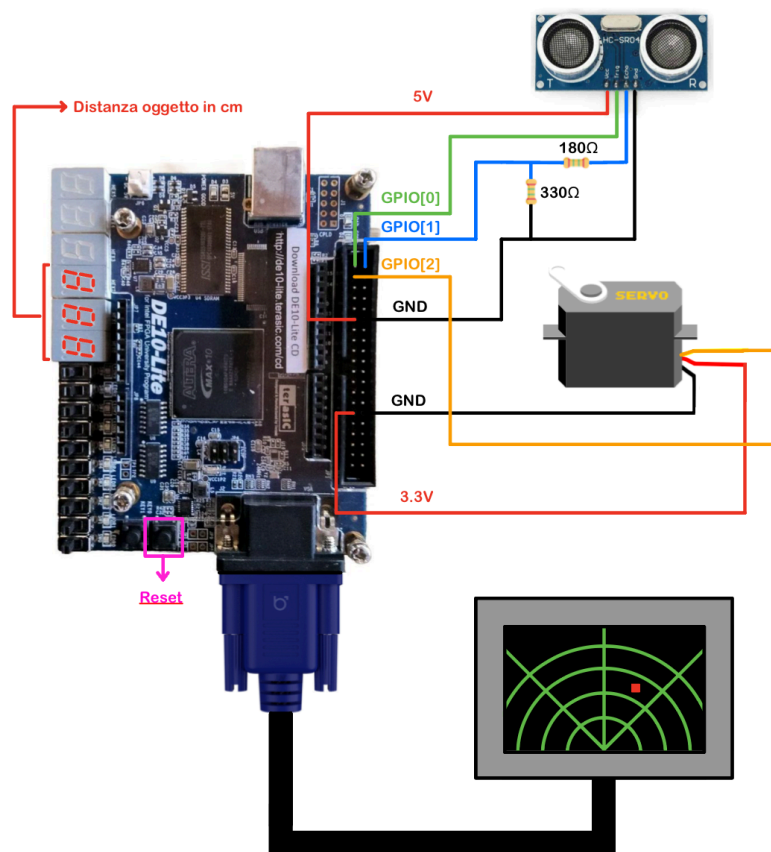
Dopodiché vengono discussi i vari step della Sintesi e del Fitting effettuati da Quartus, mettendo in evidenza l'utilizzo di risorse impiegate dal progetto. Infine viene dettagliatamente la Timing Analysis, andando a individuare il cammino critico.

## 2. Architetture

In seguito viene riportata l'interfaccia hardware dell'intero progetto e viene discussa brevemente la funzione di ogni componente. Inoltre, vengono riportate alcune strategie di progetto che sono state adottate.

### 2.1 Interfaccia Hardware

Di seguito è rappresentata l'interfaccia hardware utilizzata nel progetto, composta dai seguenti componenti:



- **Sensore HC-SR04:** Utilizzato per misurare la distanza in centimetri dell'oggetto rilevato.
- **Servomotore:** Il sensore è montato sul servomotore, che consente la rotazione da 0° a 180° e viceversa. Questa caratteristica permette di scandire l'ambiente circostante, simulando il funzionamento di un radar.
- **Display a 7 segmenti:** Impiegati per visualizzare la misura della distanza dell'oggetto rilevato in modo semplice e immediato.
- **Monitor:** Fornisce una rappresentazione grafica classica del radar, dove un quadretto rosso indica la presenza di un oggetto in una determinata posizione.

## 2.2 Strategie di Progetto

### ***Gestione segnale echo***

Poiché il sensore HC-SR04 fornisce sul segnale **Echo** un valore di **5V**, mentre la scheda accetta solo segnali a **3.3V**, è stato necessario implementare un partitore resistivo per abbassare il livello di tensione. Il partitore è stato realizzato utilizzando:

- Una resistenza da **180 Ω**
- Una resistenza da **330 Ω**

Il valore del segnale in uscita dal partitore è calcolato come:

$$V_{out} = V_{in} \cdot \frac{R_2}{R_1 + R_2}$$

dove:

- $V_{in} = 5V$
- $R_1 = 180 \Omega$
- $R_2 = 330 \Omega$

Sostituendo i valori, otteniamo:

$$V_{out} = 5 \cdot \frac{330}{180 + 330} \simeq 3.3 V$$

In questo modo, il segnale **Echo** viene adattato correttamente.

### ***Gestione del Clock***

Tutti i blocchi del sistema operano con un clock con una frequenza pari a **25 MHz**, ottenuto da un divisore di frequenza che riceve in ingresso un clock a frequenza **50 MHz** (fornito dalla DE - 10). Questo divisore dimezza la frequenza, generando un clock unico per tutto il sistema.

L'adozione di un unico clock elimina le problematiche legate alla gestione di moduli operanti a frequenze diverse, in quanto la VGA deve operare ad una frequenza pari a 25 MHz. Tramite questa soluzione si evitano violazioni di timing dovuti all'utilizzo di più domini di clock.

### ***Gestione del Reset***

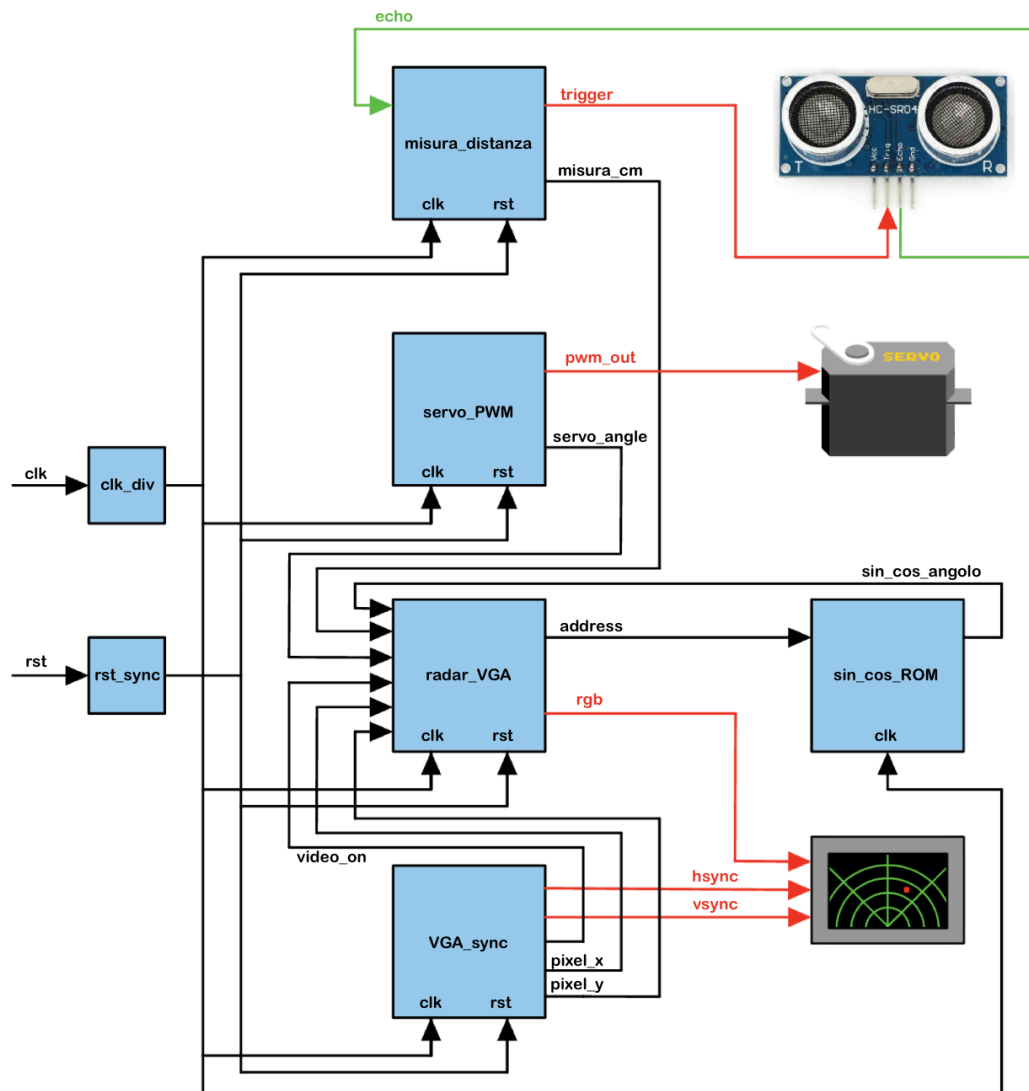
Nel progetto è stato utilizzato un reset sincrono. Tuttavia, essendo il segnale esterno di **reset** di natura asincrona, è stato sincronizzato attraverso l'utilizzo di un registro. In questo modo si evitano problemi di metastabilità da parte dei registri che campionano tale segnale.

### ***Gestione delle divisioni***

Il progetto richiede l'uso di operazioni di divisione, che sono pesanti in termini di sintesi. Per questo motivo, tali operazioni sono state sostituite con operazioni di moltiplicazione e shift.

### 3. Codice Verilog - Descrizione dei Moduli Utilizzati

Di seguito è rappresentato lo schema a blocchi che illustra le connessioni tra i vari moduli Verilog. Tutti i moduli sono stati integrati e gestiti tramite un modulo principale denominato **progetto\_PSD\_top**.



In **rosso** sono evidenziate le uscite globali del sistema.

In **verde** sono evidenziati gli ingressi globali del sistema.

In **nero** sono evidenziati i collegamenti interni tra i vari moduli.

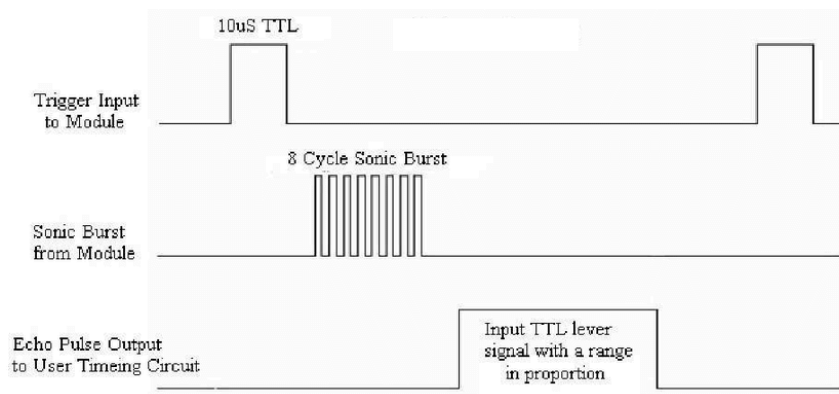
#### 3.1 misura\_distanza

Questo modulo consente, pilotando in modo opportuno il sensore HC-SR04, di ricavare la distanza a cui si trova l'oggetto rilevato. In particolare, siamo interessati

ad ottenere una misura in centimetri. Il pilotaggio del sensore segue le specifiche riportate nel datasheet, che sono di seguito descritte:

1. Fornire un impulso di **10 µs** sul pin di **Trigger**.
2. Attendere che il segnale **Echo** passi a livello alto (1), per poi iniziare a misurare il tempo.
3. Continuare a contare il tempo fino a quando il segnale **Echo** torna a livello basso (0).
4. Una volta terminato il conteggio, il sensore è pronto per una nuova misura.

A livello grafico la situazione è la seguente:



Conosciuto il tempo  $t$  misurato (in µs), è possibile calcolare la distanza tra il sensore e l'oggetto attraverso una semplice divisione.

Dato che la velocità del suono è pari a 0.034 cm/µs, la formula per calcolare la distanza è:

$$s = \frac{t \cdot 0.034}{2} = \frac{t}{58}$$

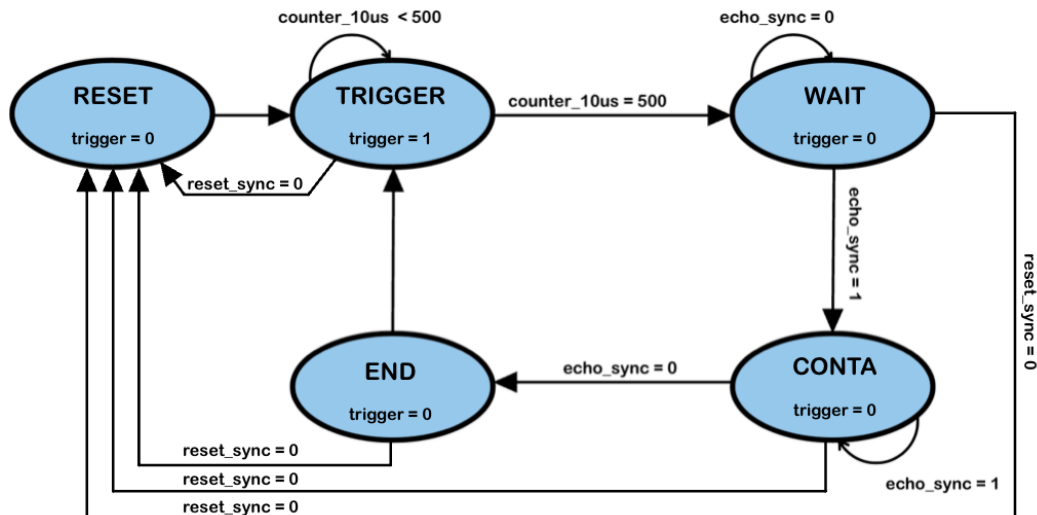
dove il fattore  $\frac{1}{2}$  tiene conto del fatto che il segnale riflesso percorre due volte la distanza (andata e ritorno). Tuttavia, poiché in Verilog le divisioni sono dispendiose in termini di risorse, si è optato per un'implementazione ottimizzata:

$$s \approx (t \cdot 2621) \gg 16$$

L'operatore " $\gg$ " si tratta dello *shift logico*; in tal caso, si tratta di una divisione per  $2^{16}$ . Questo metodo introduce un'approssimazione che, tuttavia, risulta accettabile per questo tipo di applicazione.

Il comportamento del sistema è stato implementato mediante una **macchina a stati finiti**, descritta come segue:





### Gestione glitch

Durante i test, è stato osservato un comportamento inesatto del sensore HC-SR04: dopo aver fornito una prima misura, al ciclo successivo il segnale **Echo** rimaneva per sempre basso, in seguito all'invio del segnale di **Trigger**. Questo problema causava il blocco della macchina a stati nella fase di attesa del passaggio di **Echo** a livello alto.

Per risolvere questo problema, è stato introdotto un contatore che forzi l'uscita dallo stato di attesa dopo un tempo predefinito pari a **10 µs**. In questo modo, la macchina a stati non si blocca e può continuare il suo funzionamento.

## 3.2 gestione\_misura

Il modulo **gestione\_misura** implementa una macchina a stati per il controllo e la gestione delle misure fornite dal sensore. Il modulo calcola le cifre decimali (centinaia, decine e unità) a partire dalla misura, convertita in formato binario, utilizzando l'algoritmo **Double Dabble**, che è un metodo per la conversione binario-decimale. Inoltre, le cifre calcolate vengono mostrate su tre display a 7 segmenti attraverso moduli dedicati. All'interno di questo modulo vengono quindi istanziati tre moduli **gestione\_display\_7seg**, in quanto ogni display sarà incaricato di rappresentare la corrispondente cifra decimale.

L'algoritmo Double Dabble è implementato in uno specifico ciclo for. Questo algoritmo permette di convertire un valore binario in una rappresentazione decimale utilizzando uno **shift register**. Ecco il funzionamento principale:

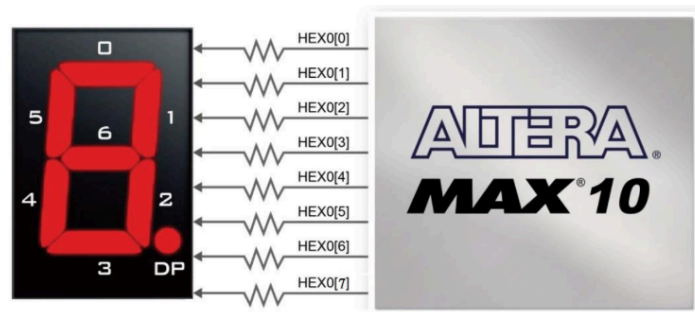
**1. Condizione di correzione:** Per ogni ciclo di shift, se una cifra decimale contenuta nel registro è maggiore o uguale a 5, viene incrementata di 3.

**2. Shift a sinistra:** Tutti i bit del registro vengono shiftati verso sinistra di una posizione per simulare un'operazione di divisione decimale.

**3. Risultato:** Dopo 9 cicli, le tre cifre decimali risultanti si trovano nelle posizioni designate del registro: shift\_reg[20:17] (centinaia), shift\_reg[16:13] (decine) e shift\_reg[12:9] (unità).

### 3.3 gestione\_display\_7seg

Questo modulo si occupa del pilotaggio del singolo display a 7 segmenti presente sul dispositivo programmabile. Il modulo riceve in ingresso la cifra decimale da rappresentare. In uscita viene fornito un vettore da 7 bit; ad ogni bit è associata l'informazione di accensione o spegnimento del relativo segmento. Questo vettore dunque indica quali segmenti del display devono essere attivati, in modo da rappresentare correttamente la cifra. Il metodo di pilotaggio del singolo display viene riportato sul manuale del dispositivo:

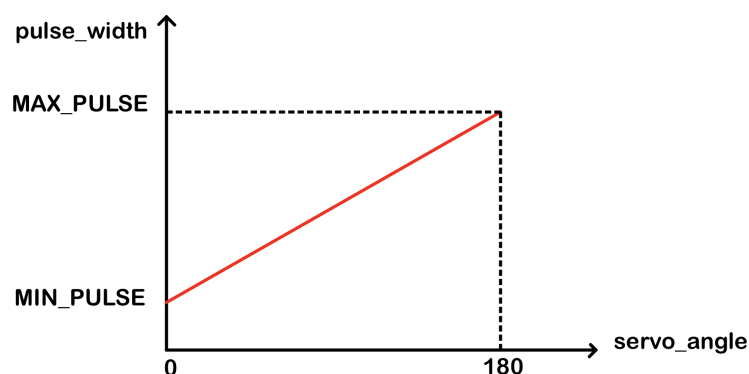


Il pilotaggio del singolo segmento è di tipo attivo basso.

### 3.4 servo\_PWM

Questo modulo genera un segnale PWM per controllare il servomotore. Questo segnale è configurato per variare la larghezza dell'impulso in funzione dell'angolo desiderato, con una mappatura lineare tra larghezza dell'impulso e posizione del servomotore. Il modulo include anche una logica per gestire il reset iniziale e per variare continuamente l'angolo del servo tra 0° e 180° con incrementi/decrementi di 2 gradi.

La mappatura viene eseguita nel seguente modo:



A 0° facciamo corrispondere MIN\_PULSE, mentre a 180° facciamo corrispondere MAX\_PULSE.

La relazione lineare è quindi la seguente:

$$pulse\_width = MIN\_PULSE + servo\_angle \cdot \frac{MAX\_PULSE - MIN\_PULSE}{180}$$

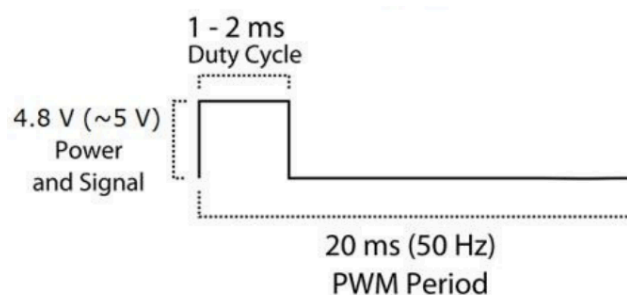
Non potendo effettuare divisioni, abbiamo leggermente modificato MIN\_PULSE e MAX\_PULSE, verificando che tale modifica non inficiasse il funzionamento del servomotore, in modo tale che si avesse:

$$\frac{MAX\_PULSE - MIN\_PULSE}{180} = 9$$

In questo modo la relazione diventa la seguente:

$$pulse\_width = MIN\_PULSE + servo\_angle \cdot 9$$

Sul datasheet è riportato il segnale per il corretto pilotaggio del servo:



Secondo il datasheet, la larghezza dell'impulso PWM deve essere:

- **1 ms** per posizionare il servo a 0°
- **2 ms** per posizionarlo a 180°

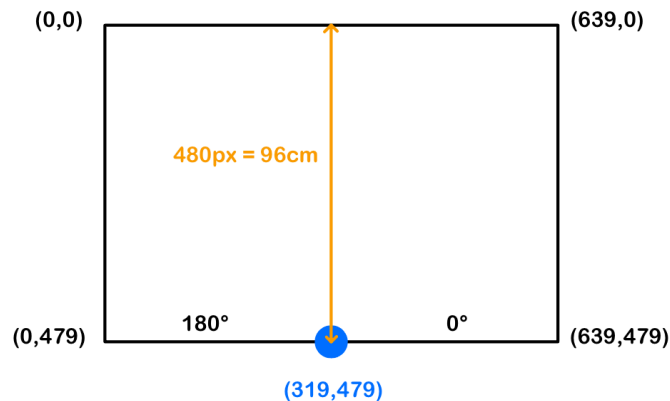
Tuttavia, durante la simulazione del modulo, si è osservato che i valori corretti per il servomotore specifico utilizzato sono:

- **400 µs** per 0°
- **2020 µs** per 180°

Questa discrepanza è stata considerata nel design del modulo per garantire un funzionamento corretto del servomotore.

### 3.5 radar\_VGA

Per il pilotaggio della **VGA**, la prima operazione è stata configurare le coordinate dell'area visibile. Tramite il modulo **VGA\_sync**, le coordinate sono state mappate in modo che il punto di origine dell'area visibile (in alto a sinistra) corrispondesse a (0,0) come mostrato in figura:



Inizialmente, il range del radar è stato limitato a **1 metro**, stabilendo una proporzione per la quale **480 pixel** corrispondessero a **1 metro**. Questo comportava la necessità di effettuare la seguente operazione:

$$modulo = \frac{distanza \cdot 480}{100}$$

Tuttavia, per evitare di effettuare una divisione è stato adottato un approccio semplificato: moltiplicare direttamente la distanza per **5**. Con questa scelta, **480 pixel** corrispondono a **96 centimetri**, che rappresenta la massima distanza visualizzabile sul monitor per un angolo di **90°**.

Dopo aver calcolato il **modulo**, ovvero la distanza in pixel alla quale si trova l'oggetto, è stato necessario determinare le coordinate del punto corrispondente sul monitor. Conoscendo l'angolo del servomotore, abbiamo applicato la conversione da coordinate polari a cartesiane utilizzando le seguenti formule:

$$obj\_x = CENTER\_X + modulo \cdot \cos(angolo)$$

$$obj\_y = CENTER\_Y - modulo \cdot \sin(angolo)$$

con:

- CENTER\_X = 319

- CENTER\_Y = 479

In modo da considerare il sensore al centro inferiore del monitor, simulando realisticamente il funzionamento di un radar. Il seno e il coseno dell'angolo si ottengono tramite la gestione di una ROM in cui tali valori sono memorizzati.

### Esempio

Immaginando che l'oggetto si trovi in una posizione di coordinate (639, 479), ovvero ad una distanza di 64 cm (infatti se 480px equivalgono a 96 cm allora 320 px equivalgono a 64 cm), e con un angolo di 0° rispetto al sensore, si ottiene:

$$modulo = distanza \cdot 5 = 64 \cdot 5 = 320$$

E quindi:

$$x_{obj} = CENTER\_X + modulo \cdot \cos(angolo) = 319 + 320 \cdot \cos(0) = 639$$

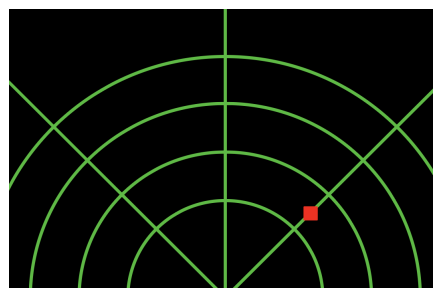
$$y_{obj} = CENTER\_Y - modulo \cdot \sin(angolo) = 479 - 320 \cdot \sin(0) = 479$$

Dopodiché si ha la **macchina a stati** per il calcolo delle coordinate dell'ostacolo. Si ha uno stato di reset seguito dallo stato *WROM*, in quanto è necessario attendere un ciclo di clock per inizializzare correttamente la ROM. In seguito, con lo stato *ADDSIN* si accede la ROM con l'indirizzo al quale è contenuto il valore del seno dell'angolo assunto dal servomotore, e con lo stato *WAITSIN* si attende che il valore venga effettivamente fornito in uscita alla memoria. Durante lo stato *SIN* si va a leggere tale valore. Lo stesso procedimento avviene per la lettura del coseno dell'angolo, il quale è contenuto nella ROM all'indirizzo al quale era contenuto il seno, sommando '181'.

Sullo schermo si rappresenta:

- 4 semicirconferenze di raggio via via crescente. Queste vengono rappresentate mediante le relative equazioni.
- 3 semirette che partono dall'origine, con pendenze rispettivamente pari a 45, 90 e 135 gradi.
- L'ostacolo rilevato dal sensore, le cui coordinate sono contenute nelle variabili *obj\_x* e *obj\_y*. L'ostacolo viene rappresentato sul display attraverso un quadrato rosso di dimensioni 5·5 pixel

L'interfaccia è quindi la seguente:



## Equazione circonferenze

Per ottenere le equazioni delle circonferenze, si utilizzano le coordinate pixel\_x e pixel\_y. L'equazione standard di una circonferenza con centro in (CENTER\_X, CENTER\_Y) è:

$$(x - CENTER\_X)^2 + (y - CENTER\_Y)^2 = r^2$$

La variabile x corrisponde a pixel\_x e la variabile y corrisponde a pixel\_y.

I valori di r scelti sono i seguenti:

- r = 100 → Punti distanti 100 pixel ovvero 20 cm dal radar
- r = 200 → Punti distanti 100 pixel ovvero 40 cm dal radar
- r = 300 → Punti distanti 100 pixel ovvero 60 cm dal radar
- r = 400 → Punti distanti 100 pixel ovvero 80 cm dal radar

## Equazione rette

Per determinare le equazioni delle rette, si adotta lo stesso approccio utilizzato per le circonferenze.

In questo caso, si considera l'equazione di una retta che passa per il punto (CENTER\_X, CENTER\_Y) e con coefficiente angolare m, che si esprime come segue:

$$y = CENTER\_Y + m \cdot (x - CENTER\_X)$$

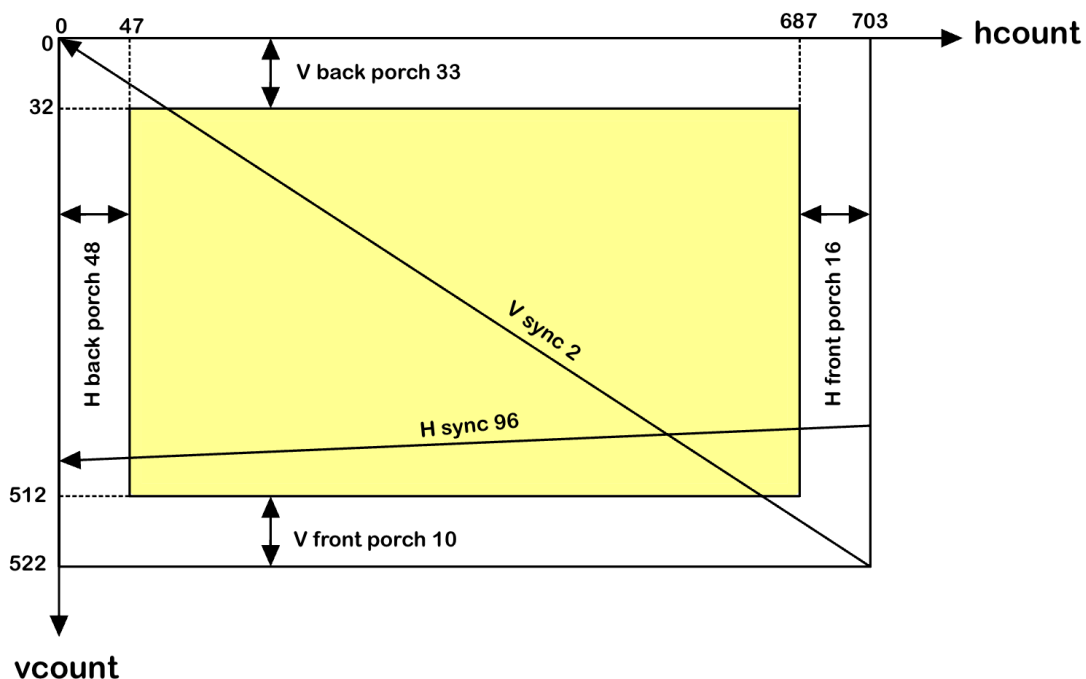
Anche in questo caso al posto di x mettiamo pixel\_x e al posto di y mettiamo pixel\_y.

Variando il valore di m è possibile ottenere le rette inclinate con l'angolo che vogliamo.

## 3.6 VGA\_sync

Il modulo **VGA\_sync** si occupa della generazione dei segnali di sincronizzazione per la VGA. Attraverso i contatori **h\_count** e **v\_count** si vanno a scandire le righe e le colonne del display. **h\_count** viene incrementato ad una frequenza di 25 MHz finché non arriva a fine riga, momento nel quale viene incrementato **v\_count**. In particolare, **h\_count** deve contare fino a 800 cicli, i quali sono dati dalla somma del numero di pixel visibili orizzontalmente (640), dalla durata di Front Porch (16 cicli) e Back Porch (48 cicli) orizzontali, e dalla durata del sync pulse orizzontale (96 cicli). Discorsi analoghi valgono per il contatore **v\_sync**, il quale deve contare 525 cicli. Il tutto viene

raffigurato nella seguente immagine, dove in giallo è stata evidenziata l'area di schermo visibile:



I parametri da considerare dunque sono i seguenti:

- H\_DISPLAY = 640
- H\_FRONT\_PORCH = 16
- H\_SYNC\_PULSE = 96
- H\_BACK\_PORCH = 48
- H\_TOTAL = 800
  
- V\_DISPLAY = 480
- V\_FRONT\_PORCH = 10
- V\_SYNC\_PULSE = 2
- V\_BACK\_PORCH = 33
- V\_TOTAL = 525

Per la corretta generazione di **hsync** (segnale attivo basso), si effettua un controllo su *h\_count*. In particolare, si abbassa se è terminato il Front Porch e lo si mantiene tale per una durata pari a *H\_SYNC\_PULSE*. Discorsi analoghi valgono per la generazione del segnale **vsync**. Successivamente si ha la gestione del segnale **video\_on**, il quale indica se ci si trova all'interno dell'area visibile.

Infine vengono generate le coordinate **pixel\_x** e **pixel\_y** per l'area visibile. Per la generazione di *pixel\_x* si verifica di trovarsi all'interno dell'area visibile e, a quel

punto, si sottrae *H\_BACK\_PORCH* (altrimenti si avrebbe un offset). Discorsi analoghi valgono per la generazione di *pixel\_y*.

### 3.7 sin\_cos ROM

Questo modulo genera un blocco di memoria ROM di dimensioni  $16 \times 512$ . Dunque è composta di 512 locazioni; ognuna di queste può contenere una parola da 16 bit. La ROM viene riempita attraverso un file .mif, il quale contiene un elenco numerato dei valori assunti da ogni singola cella, in formato binario. Si riporta in seguito la composizione della memoria ROM:

|     |                  |                   |
|-----|------------------|-------------------|
| 0   | 0000000000000000 | Valori del seno   |
| 1   | 0000001000111100 |                   |
|     | •                |                   |
|     | •                |                   |
|     | •                |                   |
| 180 | 0000000000000000 | Valori del coseno |
| 181 | 0111111111111111 |                   |
| 182 | 0111111111111010 |                   |
|     | •                |                   |
|     | •                |                   |
|     | •                |                   |
| 361 | 0000000000000000 | 0                 |
| 362 | 0000000000000000 |                   |
|     | •                |                   |
|     | •                |                   |
|     | •                |                   |
| 511 | 0000000000000000 |                   |

Nelle prime 181 celle sono memorizzati i valori del seno corrispondenti agli angoli compresi tra  $0^\circ$  e  $180^\circ$ . Le successive 181 celle contengono invece i valori del coseno per gli stessi angoli. Questi dati sono rappresentati in formato Q15, una modalità efficiente per trattare numeri frazionari. Il formato Q15 è ideale per rappresentare valori compresi tra  $-1$  e  $+1$ , come avviene per le funzioni seno e coseno. Esso utilizza 16 bit: **1 bit** è riservato al segno (complemento a due), mentre i restanti **15 bit** rappresentano la parte frazionaria.

I valori di seno e coseno, che variano nell'intervallo  $[-1, +1]$ , vengono mappati nel range  $[-32768, +32767]$  in Q15. Dunque, ciascun valore di interesse compreso tra  $-1$  e  $1$  viene moltiplicato per  $2^{15} - 1 = 32767$  per ottenere il corrispondente valore in formato Q15. Per convertire un valore Q15 al suo equivalente reale, è sufficiente dividerlo per 32768. Il vantaggio consiste nel fatto che questa operazione corrisponde alla divisione per  $2^{15}$ , implementabile con una semplice operazione di shift logico. Questo approccio consente di rappresentare con precisione i dati trigonometrici, ed evita di dover poi eseguire un'operazione di divisione.

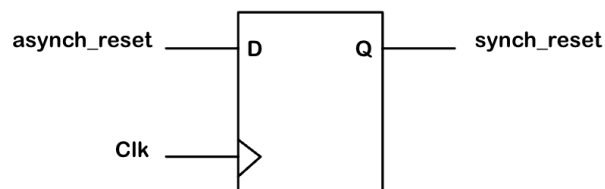


### **3.8 clk\_div**

Questo modulo si occupa della generazione del clock a 25 MHz, da inviare a tutti i blocchi sincroni della rete. Questo segnale viene ottenuto dimezzando la frequenza del clock della FPGA a 50 MHz. Dunque si tratta, a tutti gli effetti, di un divisore di frequenza.

### **3.9 reset\_sincrono**

Questo modulo si occupa della sincronizzazione del segnale di reset esterno, il quale è ovviamente asincrono (pulsante premuto dall'utente). Dunque riceve il segnale di reset asincrono in ingresso e fornisce in uscita il segnale di reset sincronizzato, il quale funge da reset globale del sistema. Si tratta, di fatto, di un flip flop di sincronizzazione.



Il segnale *synch\_reset* non fa altro che campionare il segnale *asynch\_reset*. Si avrà la copia della forma del segnale *asynch\_reset*, dove i fronti sono quantizzati al fronte di clock più vicino.

Se il segnale *asynch\_reset* dovesse violare le condizioni sui tempi di setup e di hold, il segnale *synch\_reset* andrebbe in condizioni di metastabilità. In tal caso se esce con 1 non abbiamo problemi, mentre se esce con 0 si ha un campionamento sul fronte successivo con largo margine. Si va dunque a riconoscere il fronte in ingresso sul secondo fronte disponibile.

## 4. Testbench

In seguito vengono riportati i risultati dei test, effettuati attraverso ModelSim, dei vari moduli utilizzati all'interno del progetto. Per effettuare tali test, sono stati sviluppati dei testbench.

### 4.1 Test di misura\_distanza

Questo testbench consente di verificare il corretto funzionamento del modulo che gestisce il sensore HC-SR04. Poiché non è stato possibile testare direttamente il sensore, abbiamo sviluppato un codice che genera il segnale di echo. In base alla durata del segnale di echo, il sistema calcola e fornisce le relative misure di distanza.

Sono state effettuate tre misure di seguito descritte:

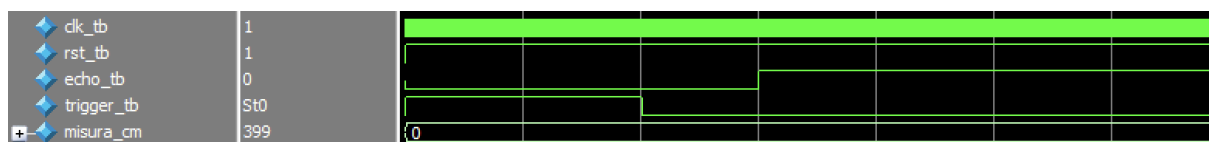
1. **Durata echo = 5800 us** →  $t(\text{us})/58 = 100 \text{ cm}$ .

2. **Durata echo = 696 us** →  $t(\text{us})/58 = 12 \text{ cm}$ .

3. **Durata echo = 23200 us** →  $t(\text{us})/58 = 400 \text{ cm}$ .

Analizziamo in dettaglio solo la prima misura, poiché le altre seguono lo stesso schema.

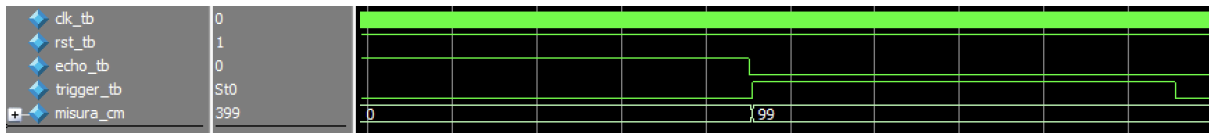
Il sistema attiva automaticamente il segnale **trigger**, impostandolo a 1 per una durata di 10  $\mu\text{s}$ , come illustrato nella seguente immagine.



Successivamente, dopo un breve intervallo di tempo introdotto manualmente per garantire che il segnale **trigger** sia tornato a basso, il segnale **echo** viene portato alto per il tempo richiesto.

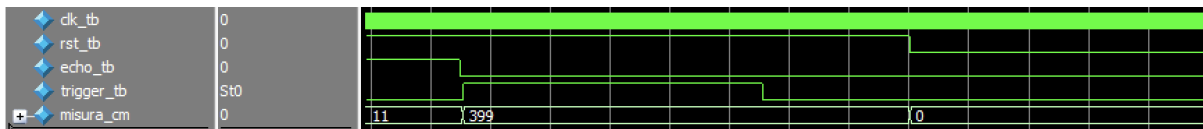
```
echo_tb = 1;  
  
#5800000; //5800us --> 1 m = 100 cm --> 01100100  
  
echo_tb = 0;
```

Al termine, il sistema calcola e restituisce la misura della distanza in centimetri, per poi riprendere con una nuova misurazione, riportando il **trigger** a 1 per altri 10  $\mu\text{s}$ .



Si osserva che il sistema non restituisce esattamente 100 cm, ma 99 cm. Questo scostamento è dovuto all'approssimazione utilizzata nelle operazioni di divisione.

Infine, l'ultimo test effettuato per questo modulo riguarda il corretto funzionamento del segnale di reset.



Come possiamo notare dopo aver calcolato la misura di 399 cm il reset viene messo a 0 (attivo basso) e *misura\_cm* va a 0.

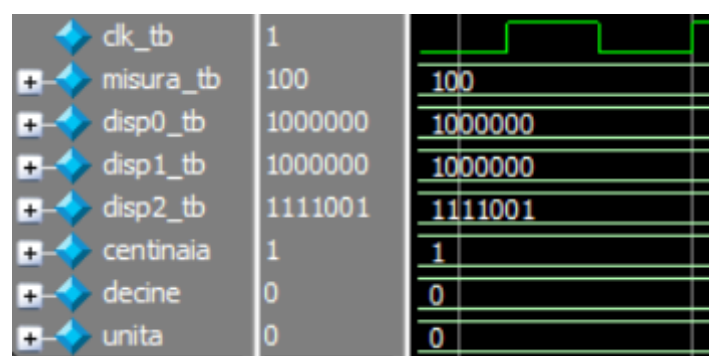
## 4.2 Test di gestione\_misura

Il seguente testbench invia al modulo **gestione\_misura** la misura effettuata dal sensore, espressa in centimetri. Si verifica dunque che modulo **gestione\_misura** generi correttamente le cifre di centinaia, decine e unità e che generi correttamente i segnali da inviare ai display (rispettivamente *disp2\_tb*, *disp1\_tb*, *disp0\_tb*). Per il test vengono inviate:

1. Misura = 100.

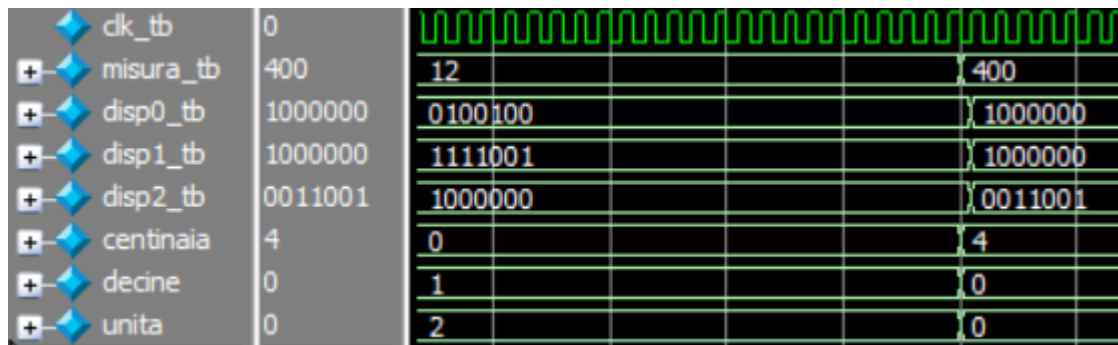
2. Misura = 12.

Le wave generate sono le seguenti:



Si osserva che le cifre assumono i valori corretti, in corrispondenza della misura di 100 cm. Il vettore di pilotaggio delle centinaia del display genera '1', mentre quelli

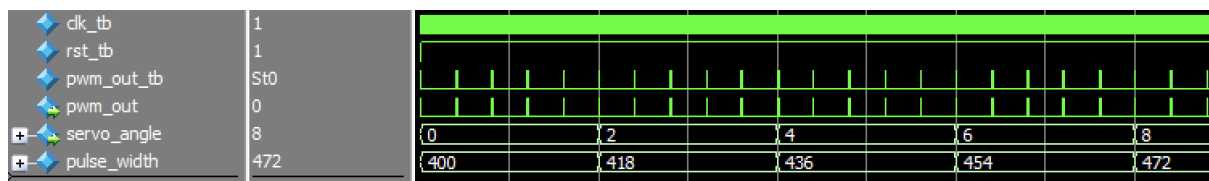
delle decine e delle unità generano entrambe '0'. A fronte di *misura\_tb* = 12, le considerazioni sono le stesse:



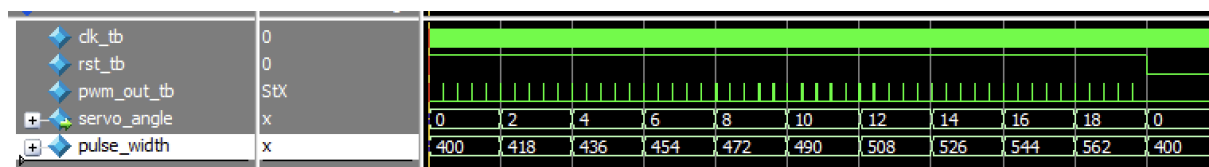
### 4.3 Test di servo\_PWM

Questo testbench consente di verificare il corretto funzionamento del modulo che pilota il servomotore. In particolare, si osserva la modifica dell'angolo con incrementi di 2° ogni 100 ms.

La variazione dell'angolo comporta anche una modifica della **pulse width**, che corrisponde alla durata del segnale **pwm\_out** e, di conseguenza, determina l'angolo al quale si posizionerà il servomotore.



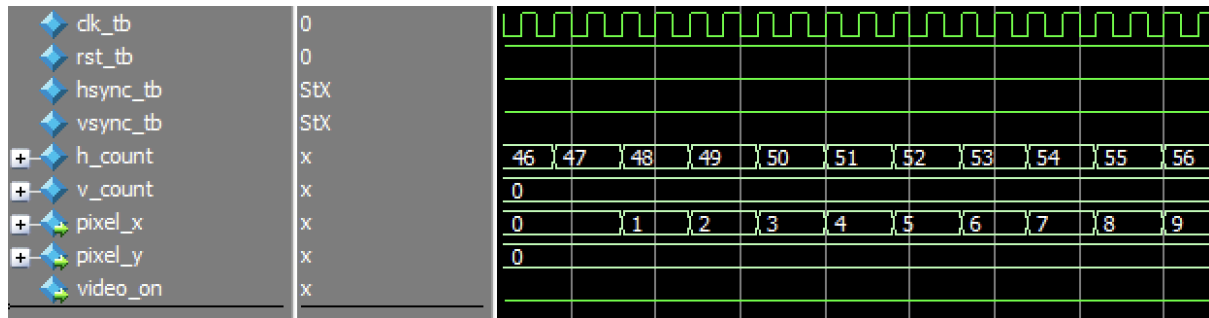
Si osserva che in corrispondenza dell'angolo 0, *pulse\_width* assume il valore di MIN\_PULSE, pari a 400. Anche in questo caso è stato verificato il corretto funzionamento del segnale di reset. Come si può osservare, quando il reset viene impostato a 0, l'angolo del servomotore torna a 0° e la **pulse width** si ripristina al valore **MIN\_PULSE**.



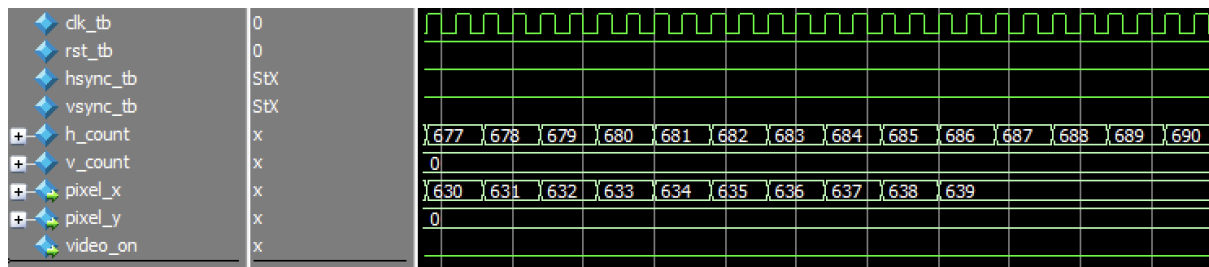
### 4.4 Test di VGA\_sync

Questo testbench consente di verificare il corretto funzionamento del modulo responsabile della generazione delle coordinate per pilotare correttamente la VGA.

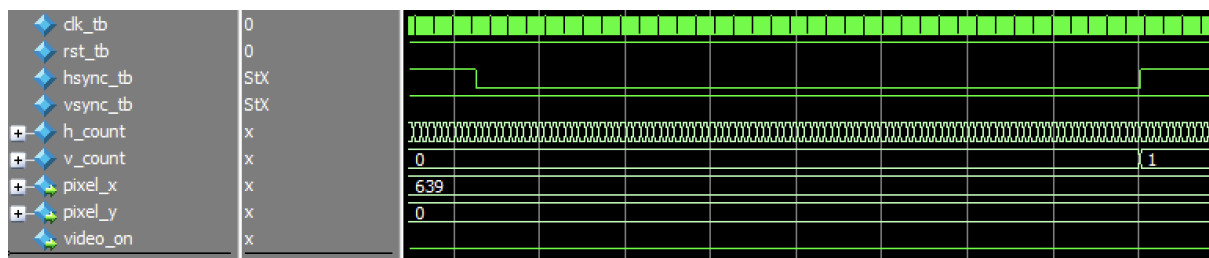
Dalla seguente immagine possiamo osservare che il segnale **pixel\_x** inizia a incrementarsi quando **h\_count** raggiunge il valore 48:



**pixel\_x** si arresta una volta arrivato a 639, mentre **h\_count** continua ad avanzare per completare le fasi di **H back porch** e **H sync**.

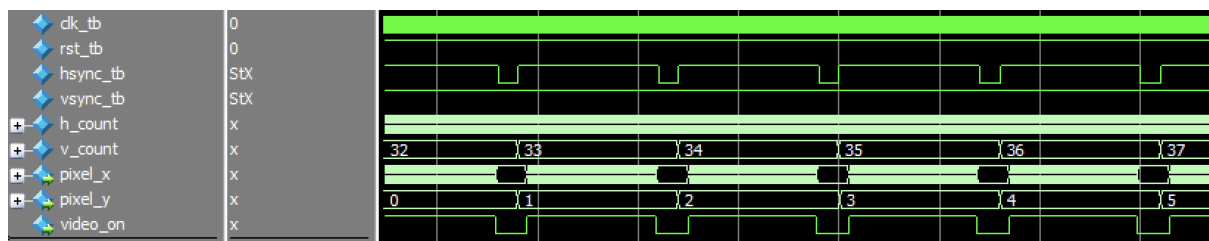


Quando **h\_count** raggiunge 703, inizia la fase di **H sync**, che dura 96 cicli di clock, durante i quali il segnale **hsync** viene portato a 0.

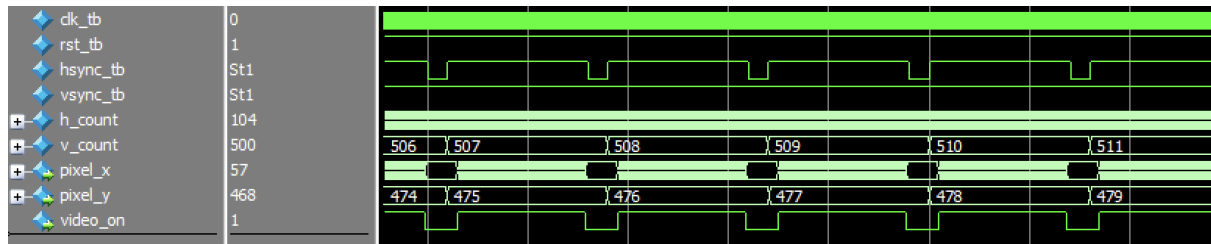


Al termine della fase di **H sync**, si passa alla riga successiva e il valore di **v\_count** viene incrementato a 1.

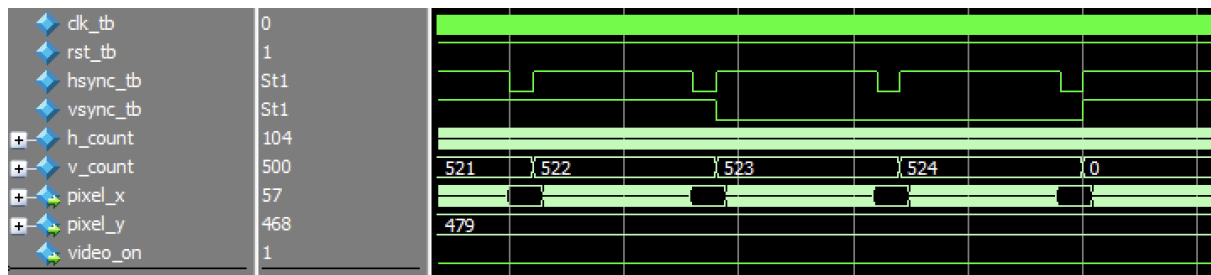
Per quanto riguarda **pixel\_y**, il suo incremento inizia quando **v\_count** raggiunge il valore 33.



**pixel\_y** si ferma quando **v\_count** arriva a 511.

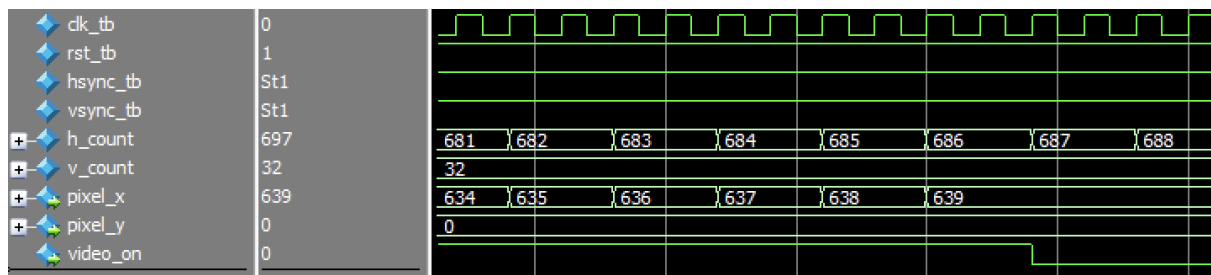
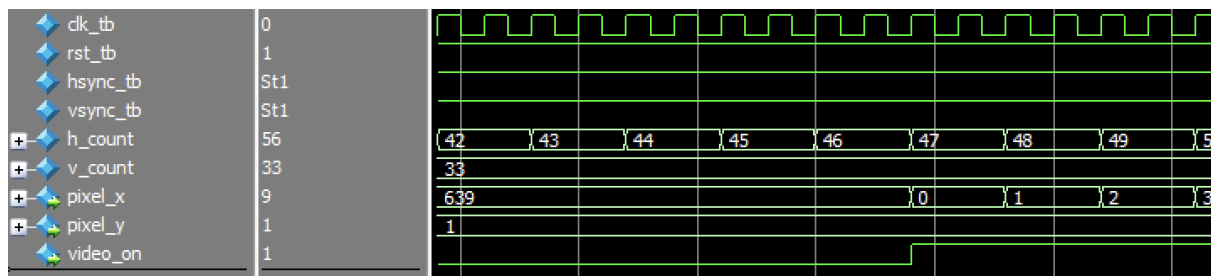


Successivamente, quando **v\_count** raggiunge il valore 523, inizia la fase di **V sync**, che dura 2 cicli di clock, durante i quali il segnale **vsync** viene portato a 0. Al termine di questa fase, **v\_count** viene riportato a 0.

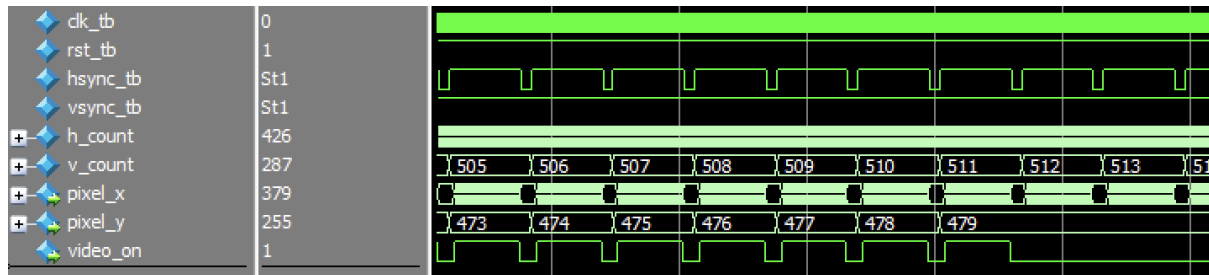
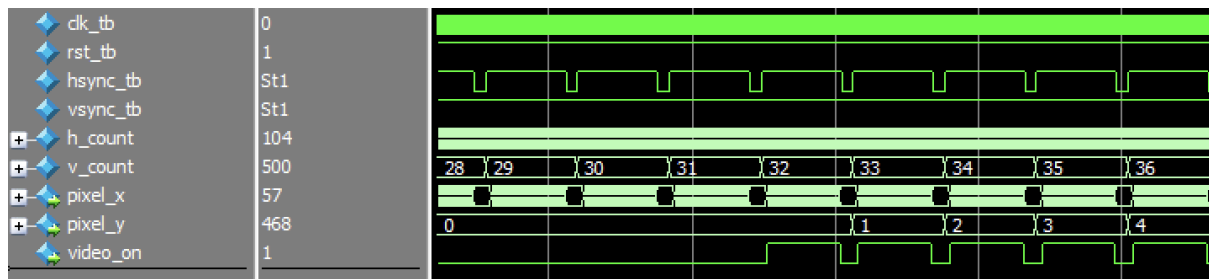


Infine, per il segnale **video\_on**, esso assume il valore 1 quando:

- **h\_count** è compreso tra 47 e 686



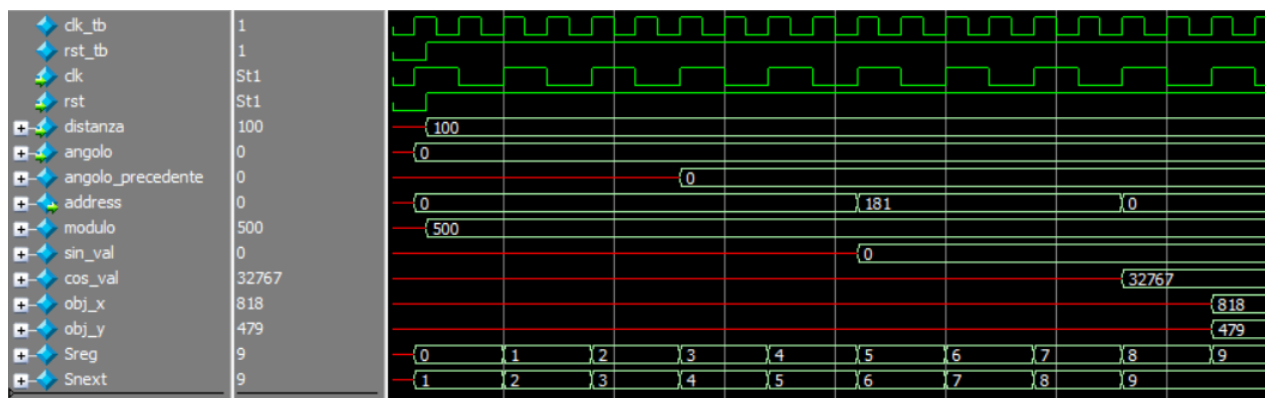
- **v\_count** è compreso tra 32 e 511.



## 4.4 Test di RADAR\_VGA

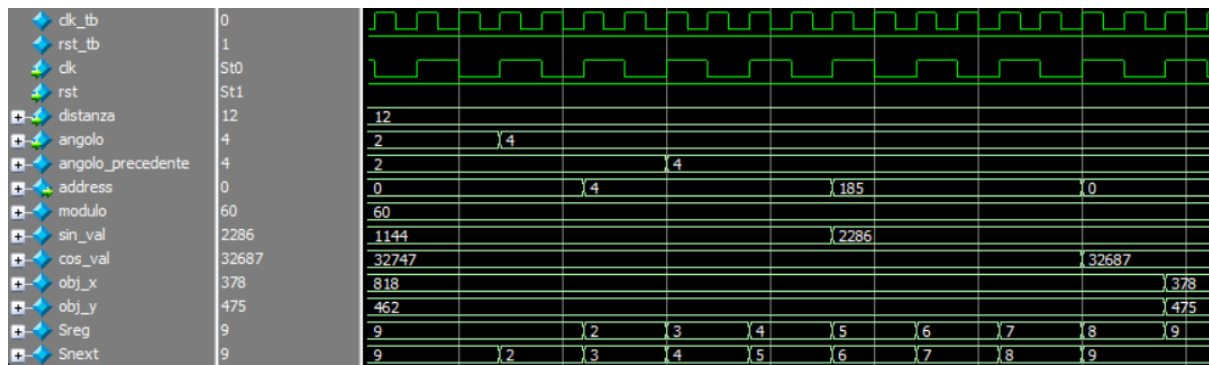
Con questo testbench si va a verificare che, in corrispondenza di una certa distanza misurata e di un certo angolo assunto dal servomotore, le coordinate X (*obj\_x*) e Y (*obj\_y*) su schermo dell'ostacolo rilevato siano corrette.

Si osserva il caso in cui **distanza = 100** e **angolo = 0**:



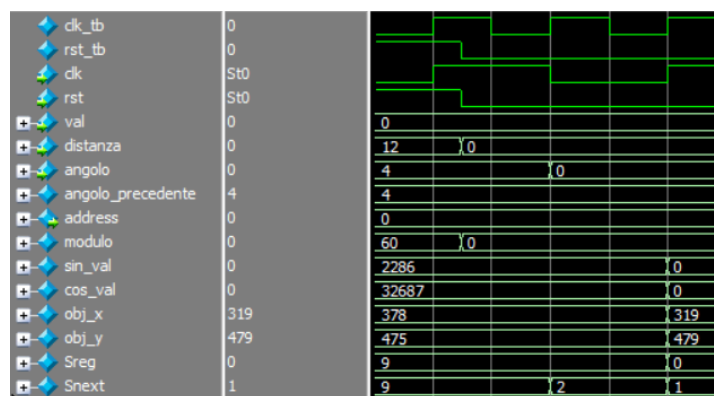
Per il completamento dell'operazione è necessario passare attraverso i 10 stati della macchina, precedentemente descritti. Il modulo viene calcolato moltiplicando *distanza* per 5, mentre *obj\_x* e *obj\_y* vengono calcolati utilizzando la relazione riportata precedentemente. Si osserva che, essendo *angolo* = 0, la coordinata Y corrisponde a CENTER\_Y. Il clock di funzionamento del modulo corrisponde a quello fornito dal test bench, diviso per 2 tramite un divisore di frequenza.

Successivamente si considera il caso **distanza = 12** e **angolo = 4**:



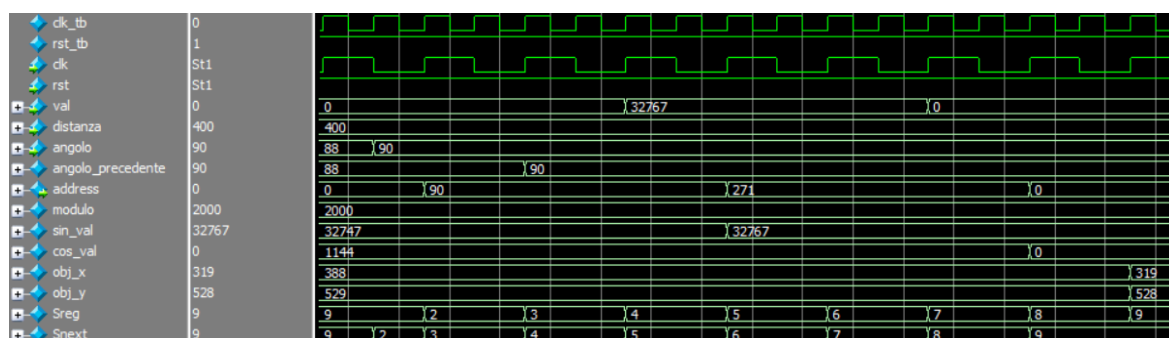
L'operazione di calcolo delle coordinate si avvia nel momento in cui *angolo* passa da 2 a 4. Vengono dunque percorsi nuovamente gli stati della macchina a stati, a meno dello stato iniziale di reset e dello stato di inizializzazione della ROM; questi servono solamente per la prima misurazione. Eccetto ciò, valgono le stesse considerazioni fatti per il caso precedente.

In caso venga ricevuto un segnale di **reset**, la situazione è la seguente:



Come si vede, al secondo fronte di clock, si entra nello stato di RESET e l'angolo del servomotore viene riportato a 0. I valori di *sin\_val* e *cos\_val* vengono portati a 0, e *obj\_x* e *obj\_y* assumono rispettivamente i valori di CENTER\_X e CENTER\_Y. Ovviamente, dato che *distanza* = 0, anche *modulo* = 0.

Viene realizzato un ultimo test bench per **misura = 400** e **angolo = 90**:





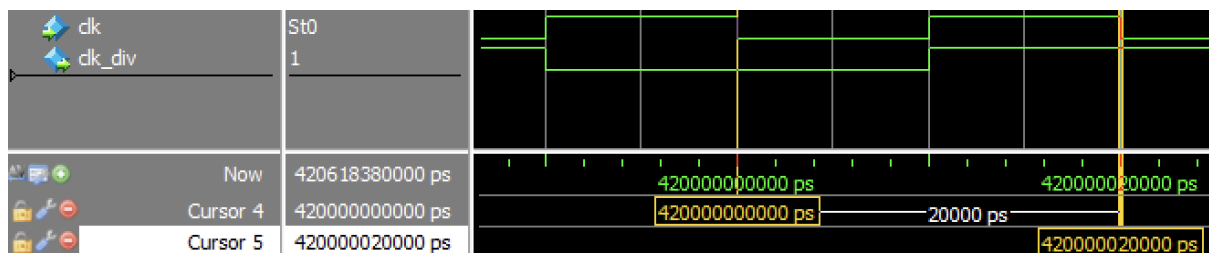
Le considerazioni sono analoghe a quelle fatte in precedenza.

## 4.5 Test di clk\_div

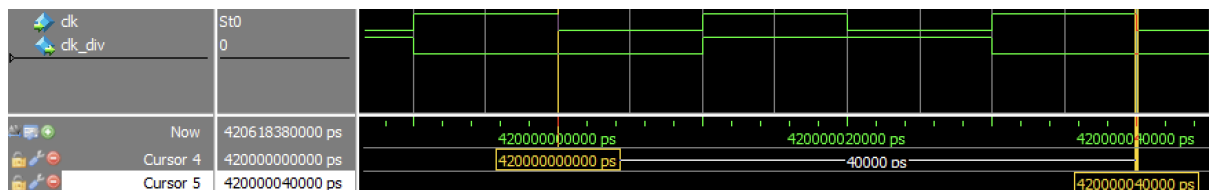
Questo testbench consente di verificare il corretto funzionamento del divisore di frequenza che, partendo da un clock a 50 MHz, genera un clock con frequenza dimezzata, pari a 25 MHz.



Di seguito è riportato il periodo del clock non diviso, ovvero di 20 ns.



Di seguito è invece riportato il periodo del clock diviso, ovvero di 40 ns.



## 4.6 Test del Reset

Questo testbench permette di verificare la corretta gestione del segnale asincrono di reset. Tale segnale, essendo generato esternamente, non è sincronizzato con il clock del sistema. Per questo motivo, viene campionato e sincronizzato tramite un flip-flop.

Come possiamo osservare nella seguente immagine, il segnale di reset asincrono viene catturato sul primo fronte utile del segnale di clock e propagato come reset\_sync. Questo processo è necessario perché, essendo il segnale asincrono, esiste la possibilità che non rispetti i tempi di setup e hold richiesti dai flip-flop nel dominio del clock.

Per risolvere questo problema, viene utilizzato un **sincronizzatore**, che garantisce che il segnale di reset venga allineato al dominio del clock del sistema, prevenendo eventuali situazioni di metastabilità.

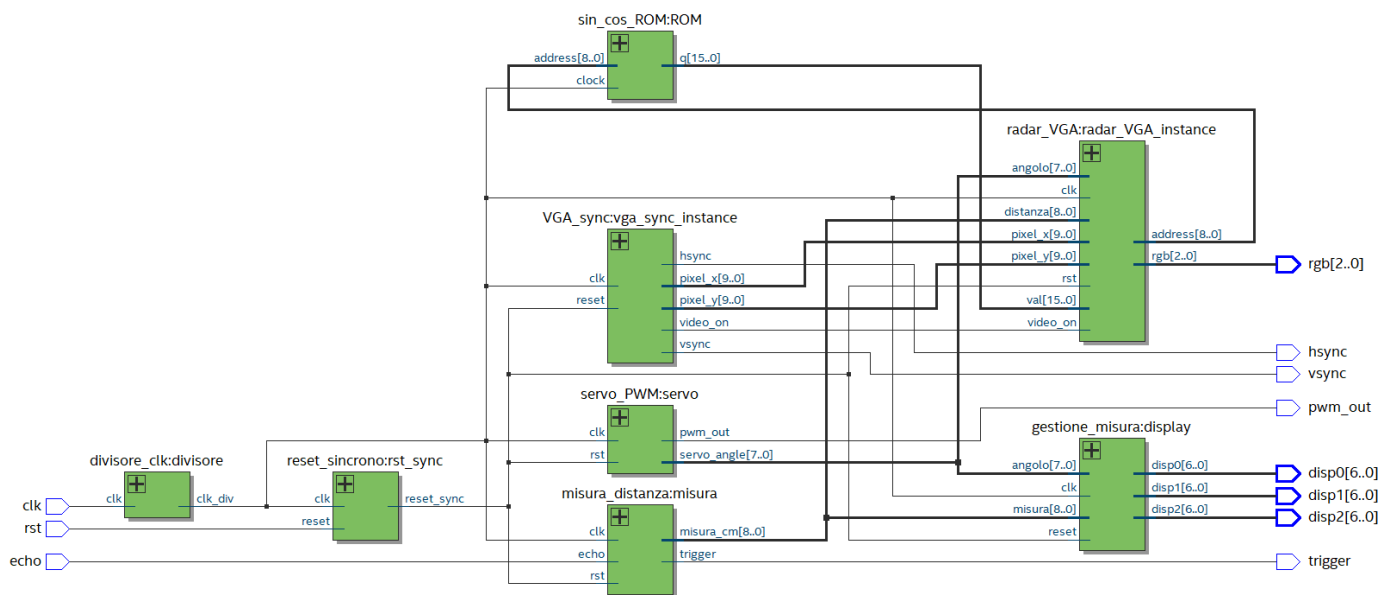


## 5. Analisi e Sintesi

Questa sezione permette di trasformare la descrizione hardware, fornita tramite linguaggio Verilog, in una rappresentazione fisica implementabile sul dispositivo FPGA. L'**analisi** si occupa della compilazione del codice, mentre la **sintesi** si occupa della traduzione del design in una rete di porte logiche.

### 5.1 RTL Viewer

Tramite l'Analisi e Sintesi è possibile visualizzare il design a livello RTL:



Si osserva la presenza dei tre ingressi e delle otto uscite della macchina. Vengono mostrati i vari moduli utilizzati, con i propri ingressi e le proprie uscite, e i collegamenti tra di essi. Questa vista consente una migliore comprensione della descrizione del circuito realizzata tramite il linguaggio Verilog. Come ci si aspettava, il clock esterno arriva al divisore di frequenza, il quale fornisce il clock diviso a tutti i moduli del sistema. Inoltre, il reset esterno arriva al sincronizzatore di reset, il quale fornisce il reset sincronizzato al resto dei moduli del sistema.

## 5.2 Summary Report

Viene mostrato il Summary Report, ottenuto attraverso l'Analisi e Sintesi:

|                                    |                                             |
|------------------------------------|---------------------------------------------|
| Analysis & Synthesis Status        | Successful - Mon Jan 27 16:04:22 2025       |
| Quartus Prime Version              | 18.1.0 Build 625 09/12/2018 SJ Lite Edition |
| Revision Name                      | PROG_VECCHIO                                |
| Top-level Entity Name              | progetto_PSD_top                            |
| Family                             | MAX 10                                      |
| Total logic elements               | 822                                         |
| Total registers                    | 287                                         |
| Total pins                         | 31                                          |
| Total virtual pins                 | 0                                           |
| Total memory bits                  | 8,192                                       |
| Embedded Multiplier 9-bit elements | 16                                          |
| Total PLLs                         | 0                                           |
| UFM blocks                         | 0                                           |
| ADC blocks                         | 0                                           |

Come riportato, il processo di sintesi ha portato all'utilizzo di **822 elementi logici**, **287 registri**, **31 pin**, e **16 moltiplicatori**. Questa si tratta di una stima delle risorse utilizzate, in quanto il design deve essere ancora mappato sul chip tramite l'operazione di Fitting.

## 5.3 Resource Usage Report

|    | Resource                                      | Usage       |
|----|-----------------------------------------------|-------------|
| 1  | Estimated Total logic elements                | 822         |
| 2  |                                               |             |
| 3  | Total combinational functions                 | 788         |
| 4  | ▼ Logic element usage by number of LUT inputs |             |
| 1  | -- 4 input functions                          | 216         |
| 2  | -- 3 input functions                          | 209         |
| 3  | -- <=2 input functions                        | 363         |
| 5  |                                               |             |
| 6  | ▼ Logic elements by mode                      |             |
| 1  | -- normal mode                                | 372         |
| 2  | -- arithmetic mode                            | 416         |
| 7  |                                               |             |
| 8  | ▼ Total registers                             | 287         |
| 1  | -- Dedicated logic registers                  | 287         |
| 2  | -- I/O registers                              | 0           |
| 9  |                                               |             |
| 10 | I/O pins                                      | 31          |
| 11 | Total memory bits                             | 8192        |
| 12 |                                               |             |
| 13 | Embedded Multiplier 9-bit elements            | 16          |
| 14 |                                               |             |
| 15 | Maximum fan-out node                          | divi..._div |
| 16 | Maximum fan-out                               | 303         |
| 17 | Total fan-out                                 | 3473        |
| 18 | Average fan-out                               | 2.97        |

Viene riportato un report più dettagliato sull'utilizzo delle risorse (elementi logici, registri...), in seguito all'operazione di Analisi e Sintesi. Vengono fornite anche informazioni sul **fan-out** totale e medio della rete; si osserva che il divisore del clock è il blocco della rete con un fan-out maggiore.

## 6. Fitting - Place and Route

Questa sezione si occupa della fase di mappatura e posizionamento delle risorse fisiche sul dispositivo FPGA. In questa fase, il design sintetizzato viene trasformato in una disposizione ottimizzata di celle logiche e interconnessioni. Il processo di "Place" definisce la posizione delle risorse all'interno del chip, mentre il "Route" si occupa della connessione tra queste risorse.

### 6.1 Fitter Summary

Dato che è stata effettuata l'operazione di Fitting, si ottiene un nuovo report. Questo, a differenza di quello fornito al termine dell'Analisi e Sintesi, riporta il numero effettivo di risorse fisiche utilizzate dall'FPGA. Infatti, a questo punto, le risorse sono state allocate fisicamente all'interno del chip. Il Summary Report ottenuto è il seguente:

|                                    |                                             |
|------------------------------------|---------------------------------------------|
| Fitter Status                      | Successful - Mon Jan 27 16:04:35 2025       |
| Quartus Prime Version              | 18.1.0 Build 625 09/12/2018 SJ Lite Edition |
| Revision Name                      | PROG_VECCHIO                                |
| Top-level Entity Name              | progetto_PSD_top                            |
| Family                             | MAX 10                                      |
| Device                             | 10M50DAF484C7G                              |
| Timing Models                      | Final                                       |
| Total logic elements               | 821 / 49,760 ( 2 % )                        |
| Total registers                    | 255                                         |
| Total pins                         | 31 / 360 ( 9 % )                            |
| Total virtual pins                 | 0                                           |
| Total memory bits                  | 8,192 / 1,677,312 ( < 1 % )                 |
| Embedded Multiplier 9-bit elements | 16 / 288 ( 6 % )                            |
| Total PLLs                         | 0 / 4 ( 0 % )                               |
| UFM blocks                         | 0 / 1 ( 0 % )                               |
| ADC blocks                         | 0 / 2 ( 0 % )                               |

Si osserva che sono stati utilizzati 821 **logic elements** su un totale di 49.760 disponibili, pari al 2% delle risorse dell'FPGA. Il progetto fa uso di 255 **registri** e impiega 31 **pin fisici** del dispositivo su 360 totali, raggiungendo un utilizzo del 9%. Non sono stati utilizzati **virtual pin**, mentre il consumo di **memoria** ammonta a 8.192 bit su 1.677.312 disponibili, un valore inferiore all'1%. Inoltre, il progetto sfrutta 16 **moltiplicatori** a 9 bit su 288 disponibili, rappresentando un utilizzo pari al 6% delle risorse destinate a tale funzione.

Come detto in precedenza i dati riportati sono più precisi e riflettono l'effettivo utilizzo delle risorse dopo il "place and routing".

## 6.2 Resource Usage Summary

|    | Resource                                      | Usage                       |
|----|-----------------------------------------------|-----------------------------|
| 1  | ✓ Total logic elements                        | 821 / 49,760 ( 2 % )        |
| 1  | -- Combinational with no register             | 566                         |
| 2  | -- Register only                              | 13                          |
| 3  | -- Combinational with a register              | 242                         |
| 2  |                                               |                             |
| 3  | ✓ Logic element usage by number of LUT inputs |                             |
| 1  | -- 4 input functions                          | 216                         |
| 2  | -- 3 input functions                          | 209                         |
| 3  | -- <=2 input functions                        | 383                         |
| 4  | -- Register only                              | 13                          |
| 4  |                                               |                             |
| 5  | ✓ Logic elements by mode                      |                             |
| 1  | -- normal mode                                | 392                         |
| 2  | -- arithmetic mode                            | 416                         |
| 6  |                                               |                             |
| 7  | ✓ Total registers*                            | 255 / 51,509 ( < 1 % )      |
| 1  | -- Dedicated logic registers                  | 255 / 49,760 ( < 1 % )      |
| 2  | -- I/O registers                              | 0 / 1,749 ( 0 % )           |
| 8  |                                               |                             |
| 9  | Total LABs: partially or completely used      | 71 / 3,110 ( 2 % )          |
| 10 | Virtual pins                                  | 0                           |
| 11 | ✓ I/O pins                                    | 31 / 360 ( 9 % )            |
| 1  | -- Clock pins                                 | 2 / 8 ( 25 % )              |
| 2  | -- Dedicated input pins                       | 1 / 1 ( 100 % )             |
| 12 |                                               |                             |
| 13 | M9Ks                                          | 1 / 182 ( < 1 % )           |
| 14 | UFM blocks                                    | 0 / 1 ( 0 % )               |
| 15 | ADC blocks                                    | 0 / 2 ( 0 % )               |
| 16 | Total block memory bits                       | 8,192 / 1,677,312 ( < 1 % ) |
| 17 | Total block memory implementation bits        | 9,216 / 1,677,312 ( < 1 % ) |
| 18 | Embedded Multiplier 9-bit elements            | 16 / 288 ( 6 % )            |
| 19 | PLLs                                          | 0 / 4 ( 0 % )               |
| 20 | ✓ Global signals                              | 1                           |
| 1  | -- Global clocks                              | 1 / 20 ( 5 % )              |
| 21 | JTAGs                                         | 0 / 1 ( 0 % )               |
| 22 | CRC blocks                                    | 0 / 1 ( 0 % )               |
| 23 | Remote update blocks                          | 0 / 1 ( 0 % )               |
| 24 | Oscillator blocks                             | 0 / 1 ( 0 % )               |
| 25 | Impedance control blocks                      | 0 / 1 ( 0 % )               |
| 26 | Average interconnect usage (total/H/V)        | 0.5% / 0.5% / 0.5%          |
| 27 | Peak interconnect usage (total/H/V)           | 6.2% / 6.3% / 6.1%          |
| 28 | Maximum fan-out                               | 261                         |
| 29 | Highest non-global fan-out                    | 149                         |
| 30 | Total fan-out                                 | 3276                        |
| 31 | Average fan-out                               | 2.80                        |

Questa tabella mostra una panoramica più approfondita delle risorse utilizzate dall'FPGA dopo il processo di fitting. Degli 821 elementi logici utilizzati, 566 sono elementi combinatori senza registri, 13 solo registri, e 242 combinano logica combinatoria e registri. Viene inoltre riportato il numero di elementi logici utilizzati, a seconda del numero di input delle LUT impiegate.

Dal punto di vista della modalità operativa, 392 logic elements funzionano in modalità normale, mentre 416 sono configurati in modalità aritmetica. L'utilizzo dei

registri ammonta a 255 unità su 51.509 disponibili, concentrandosi esclusivamente sui registri logici dedicati. Non vengono impiegati registri I/O.

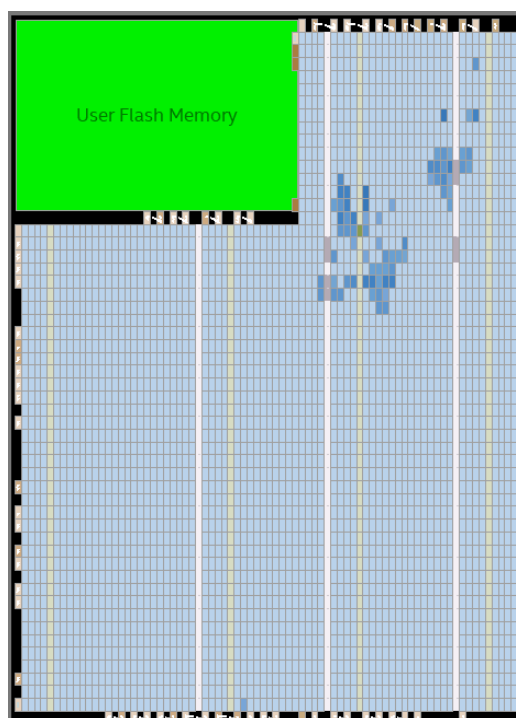
Per quanto riguarda i pin, vengono utilizzati 31 pin fisici su 360 disponibili, con un impiego del 9%. Tra questi, 2 sono clock pin (25% di utilizzo), mentre 1 è dedicato esclusivamente all'ingresso.

L'utilizzo della memoria è limitato: il progetto impiega 8.192 bit di memoria su un massimo di 1.677.312, ovvero meno dell'1%. Vengono utilizzati 16 moltiplicatori a 9 bit su 288 disponibili, pari a un utilizzo del 6%. Nessun PLL, blocco UFM o ADC viene utilizzato.

La sezione relativa al fan-out mostra un valore massimo di 261, con un fan-out medio pari a 2.80.

## **6.3 Layout**

Viene mostrato il Layout del dispositivo FPGA:



Nell'immagine viene posto in evidenza dove sono posizionate le risorse utilizzate nel progetto. La griglia celeste rappresenta l'area totale del chip, suddivisa in blocchi logici. Le celle più scure indicano le regioni effettivamente utilizzate dal progetto. L'uso delle risorse è più o meno concentrato in alcune aree specifiche del dispositivo; ciò è probabilmente dovuto ad un'ottimizzazione che consente di minimizzare il routing.



## 7. Timing Analysis

Vengono riportati i risultati della Timing Analysis. Questa consente di trovare la massima frequenza di clock a cui può operare correttamente il sistema. Inoltre, grazie a questo tipo di analisi si riescono a valutare gli **slack di setup** e di **hold**; se questi risultano positivi, non si hanno setup e hold violation. Infine viene discusso e visualizzato il path critico, che è quello che definisce la **massima frequenza di clock**.

### 7.1 Constraints di Clock

Viene riportato il file **.sdc**, grazie al quale vengono specificate le constraints di clock:

```
set_time_format -unit ns -decimal_places 3
create_clock -name {clk} -period 20.000 -waveform { 0.000 10.000 } [get_ports {clk}]
create_generated_clock -name clk_div -divide_by 2 -source clk divisore_clk:divisore|clk_div
```

Viene creato il clock principale, denominato **clk**, con un periodo di 20 nanosecondi e un duty cycle del 50%. Dopodiché viene generato un secondo clock, chiamato **clk\_div**, che è proprio il clock che ricevono i moduli del progetto. Questo clock viene creato a partire dal clock principale, dimezzando la frequenza.

Dunque, al termine della Timing Analysis, si verifica che i due clock siano stati riconosciuti correttamente:

| Clock Name | Type    | Period | Frequency | Rise  | Fall   | Duty Cycle | Divide by | Multiply by |
|------------|---------|--------|-----------|-------|--------|------------|-----------|-------------|
| clk        | Base    | 20.000 | 50.0 MHz  | 0.000 | 10.000 |            |           |             |
| clk_div    | Ge...ed | 40.000 | 25.0 MHz  | 0.000 | 20.000 |            | 2         | 1           |

Effettivamente, i due clock sono stati riconosciuti ed hanno le caratteristiche desiderate. Il Timing Analyzer fornisce la frequenza massima alla quale può funzionare il sistema:

| Fmax      | Restricted Fmax | Clock Name | Note |
|-----------|-----------------|------------|------|
| 61.92 MHz | 61.92 MHz       | clk_div    |      |

Dunque ci si può spingere fino ad avere un clock di frequenza pari a **61.92 MHz**. A questo punto si va a verificare che entrambi gli slack ottenuti siano positivi.

## 7.2 Slack di Setup

Viene riportato lo slack di setup:

| Clock   | Slack  | End Point TNS |
|---------|--------|---------------|
| clk     | 19.298 | 0.000         |
| clk_div | 23.851 | 0.000         |

Concentrandosi su **clk\_div**, che è proprio il clock con il quale vengono sincronizzati tutti i moduli del sistema, si osserva uno slack di setup positivo pari a **23.851 ns**, che ci assicura del fatto che non si abbiano violazioni sui tempi di setup.

## 7.3 Slack di Hold

Viene riportato lo slack di hold:

| Clock   | Slack | End Point TNS |
|---------|-------|---------------|
| clk_div | 0.341 | 0.000         |
| clk     | 0.393 | 0.000         |

Anche in questo caso, si ottiene uno slack positivo. In particolare, assume un valore di **0.341 ns**. In questo caso, dunque, viene garantito che non si abbiano violazioni sui tempi di hold.

## 7.4 Summary of Paths

Attraverso la seguente sezione del Timing Analyzer, si individuano i 10 path più critici e il relativo slack:

|    | Slack  | From Node                            | To Node                             | Launch Clock | Latch Clock | Relationship | Clock Skew | Data Delay |
|----|--------|--------------------------------------|-------------------------------------|--------------|-------------|--------------|------------|------------|
| 1  | 23.851 | VGA_syncvga_sync...stance pixel_x[2] | radar_VGA:radar_VGA_instance rgb[1] | clk_div      | clk_div     | 40.000       | -0.064     | 16.098     |
| 2  | 23.888 | VGA_syncvga_sync...stance pixel_x[0] | radar_VGA:radar_VGA_instance rgb[1] | clk_div      | clk_div     | 40.000       | -0.066     | 16.059     |
| 3  | 23.896 | VGA_syncvga_sync...stance pixel_x[2] | radar_VGA:radar_VGA_instance rgb[1] | clk_div      | clk_div     | 40.000       | -0.064     | 16.053     |
| 4  | 23.933 | VGA_syncvga_sync...stance pixel_x[0] | radar_VGA:radar_VGA_instance rgb[1] | clk_div      | clk_div     | 40.000       | -0.066     | 16.014     |
| 5  | 23.938 | VGA_syncvga_sync...stance pixel_x[4] | radar_VGA:radar_VGA_instance rgb[1] | clk_div      | clk_div     | 40.000       | -0.064     | 16.011     |
| 6  | 23.977 | VGA_syncvga_sync...stance pixel_x[2] | radar_VGA:radar_VGA_instance rgb[1] | clk_div      | clk_div     | 40.000       | -0.064     | 15.972     |
| 7  | 23.983 | VGA_syncvga_sync...stance pixel_x[4] | radar_VGA:radar_VGA_instance rgb[1] | clk_div      | clk_div     | 40.000       | -0.064     | 15.966     |
| 8  | 23.985 | VGA_syncvga_sync...stance pixel_x[3] | radar_VGA:radar_VGA_instance rgb[1] | clk_div      | clk_div     | 40.000       | -0.064     | 15.964     |
| 9  | 23.995 | VGA_syncvga_sync...stance pixel_x[3] | radar_VGA:radar_VGA_instance rgb[1] | clk_div      | clk_div     | 40.000       | -0.064     | 15.954     |
| 10 | 24.001 | VGA_syncvga_sync...stance pixel_x[6] | radar_VGA:radar_VGA_instance rgb[1] | clk_div      | clk_div     | 40.000       | -0.064     | 15.948     |

Il **path critico** è il primo nell'elenco riportato. Infatti ad esso è associato lo slack minimo, che è proprio quello riportato precedentemente. Dunque si tratta del cammino che limita la frequenza massima di funzionamento del sistema.

## 7.5 Data Arrival Path e Data required path

Il **Data Arrival Path** rappresenta il percorso che un segnale dati segue dal momento in cui viene lanciato fino a quando raggiunge la destinazione, ovvero il registro che lo deve acquisire. Questo percorso, come è possibile notare dalla tabella seguente, è composto da più segmenti, ognuno dei quali introduce un certo ritardo dovuto alla propagazione del segnale attraverso la rete logica.

| Data Arrival Path |          |        |    |      |        |          |                  |
|-------------------|----------|--------|----|------|--------|----------|------------------|
|                   | Total    | Incr   | RF | Type | Fanout | Location | Element          |
| 1                 | 0.000    | 0.000  |    |      |        |          | launch edge time |
| 2                 | > 6.808  | 6.808  |    |      |        |          | clock path       |
| 3                 | > 22.906 | 16.098 |    |      |        |          | data path        |

Il primo elemento rappresenta il *launch edge time*, ovvero il momento in cui il segnale viene lanciato, con un ritardo iniziale pari a 0. Successivamente, il *clock path* (percorso seguito dal segnale di clock per raggiungere gli elementi di sincronizzazione) introduce un ritardo di 6.808 ns, determinato dal percorso del segnale di clock all'interno del circuito. Infine, il *data path* (percorso che il segnale dati percorre dalla sua origine fino alla destinazione, passando attraverso la logica combinatoria e le connessioni fisiche del circuito) contribuisce con un ulteriore ritardo di 16.098 ns, portando il ritardo totale del percorso dati a 22.906 ns.

Il **Data Required Path** rappresenta il tempo massimo entro cui un segnale dati deve raggiungere la destinazione, affinché venga acquisito correttamente dal registro di destinazione. Anche questo percorso è composto da più segmenti, ognuno dei quali introduce un certo ritardo:

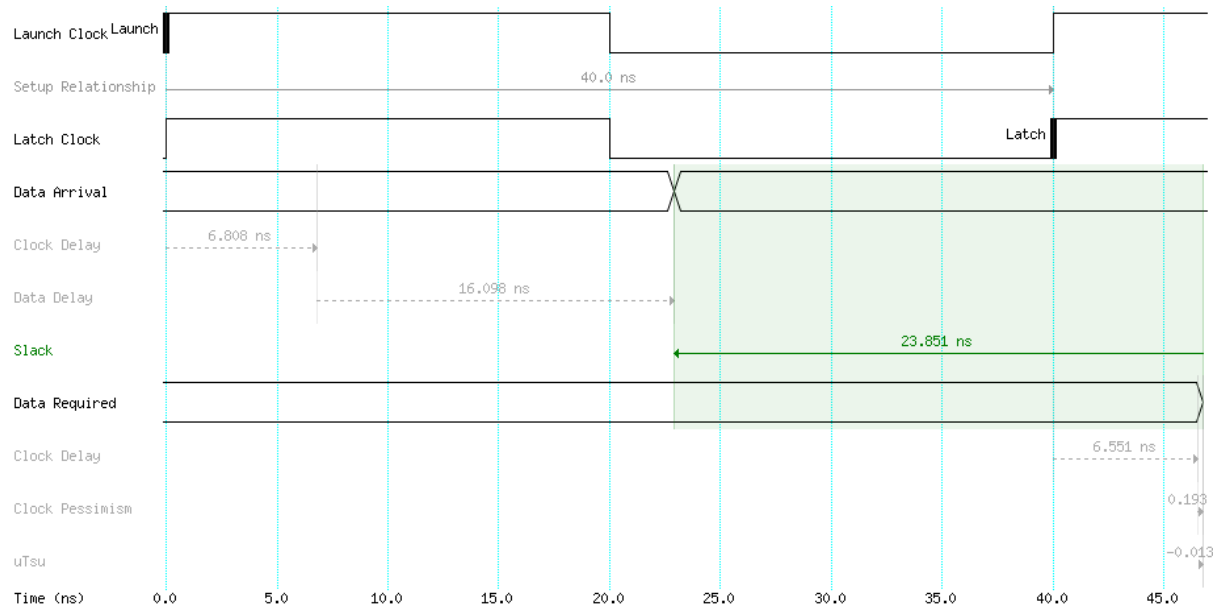
| Data Required Path |          |        |    |      |        |                |                                     |
|--------------------|----------|--------|----|------|--------|----------------|-------------------------------------|
|                    | Total    | Incr   | RF | Type | Fanout | Location       | Element                             |
| 1                  | 40.000   | 40.000 |    |      |        |                | latch edge time                     |
| 2                  | > 46.744 | 6.744  |    |      |        |                | clock path                          |
| 3                  | 46.757   | 0.013  |    | uTsu | 1      | FF_X50_Y33_N19 | radar_VGA:radar_VGA_instance rgb[1] |

Il primo valore corrisponde al *latch edge time*, ovvero il tempo di riferimento in cui il segnale deve essere acquisito, fissato a 40.000 ns. Successivamente, il *clock path* introduce un ritardo di 6.744 ns, che rappresenta il tempo impiegato dal segnale di clock per raggiungere il registro di arrivo. Infatti, l'effettivo istante di campionamento da parte del Latch FF avviene con un ritardo dovuto al clock path. Infine, l'ultima voce nella tabella fa riferimento a un ritardo aggiuntivo di 0.013 ns associato al vincolo di *setup time* (uTsu), che rappresenta il tempo minimo necessario affinché il dato venga stabilizzato prima del fronte di clock. Il valore totale alla fine del percorso risulta essere di 46.757 ns.

Si può notare che il valore di clock path nei due casi risulta essere diverso. Infatti, nel **Data Arrival Path**, il *clock path* rappresenta il ritardo accumulato dal segnale di clock nella fase di lancio (*launch clock*), ovvero il tempo impiegato per attivare il registro sorgente da cui parte il dato. Nel **Data Required Path**, invece, il *clock path*

si riferisce al ritardo del segnale di clock nella fase di acquisizione (*latch clock*), cioè il tempo impiegato per raggiungere il registro di destinazione.

E' possibile visualizzare il tutto da un punto di vista grafico con la seguente temporizzazione:



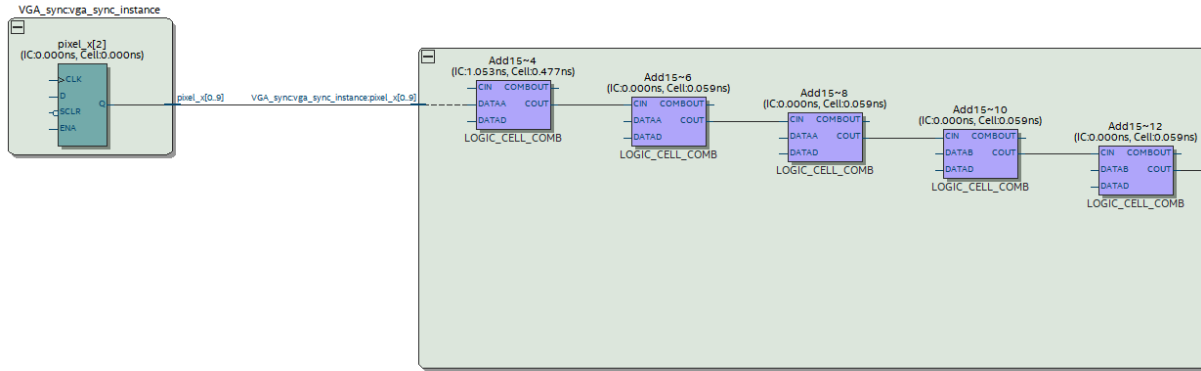
Questa mostra i ritardi lungo il percorso del segnale, consentendo di verificare se i requisiti di timing sono soddisfatti. In particolare, il Launch Clock rappresenta il fronte di clock che arriva al registro di partenza, mentre il Latch Clock è il clock che va al registro di arrivo. Il tempo necessario affinché il segnale dati arrivi alla destinazione (Data Arrival) include tutti i ritardi precedentemente descritti. Il Data Required è l'istante in cui i dati devono essere validi affinché il campionamento possa avvenire senza errori.

La quantità di tempo disponibile per garantire che i dati arrivino in tempo si tratta proprio dello Slack, che in questo caso è pari a 23.851 ns, come visto in precedenza. Questo valore positivo indica che c'è un margine di sicurezza tra l'arrivo effettivo del dato e il momento in cui il dato è richiesto, dimostrando che il percorso soddisfa i vincoli di timing.

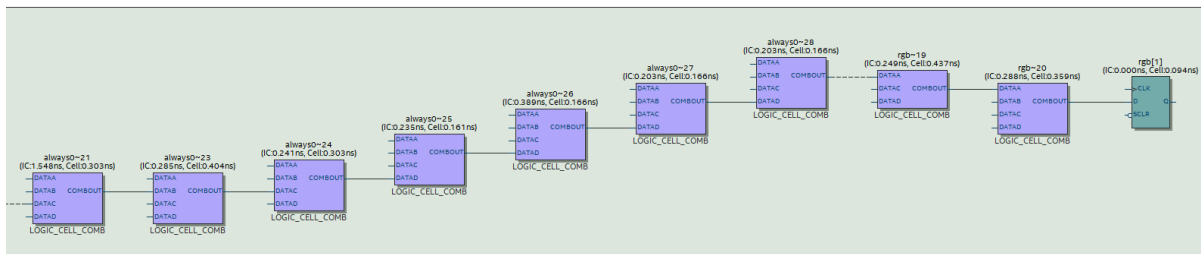
## 7.6 Path in technology map viewer

Vengono presentate alcune immagini relative al percorso critico all'interno del design. Queste immagini mostrano in dettaglio i percorsi temporali analizzati, evidenziando le connessioni tra i vari componenti del circuito.

Il path critico parte dal registro **VGA\_syncvga\_sync\_instance**. Questo è seguito da una serie di celle logiche, come si vede in figura:



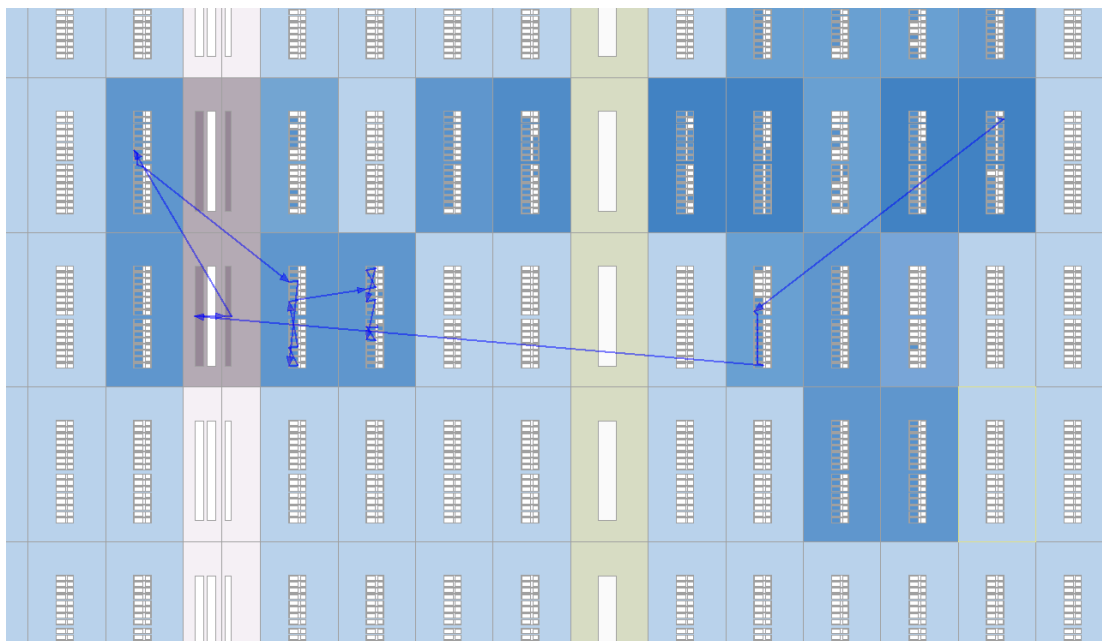
La parte finale evidenzia un'ulteriore successione di celle logiche. Questi blocchi convergono nell'ultimo elemento, che corrisponde al registro **rgb[1]**:



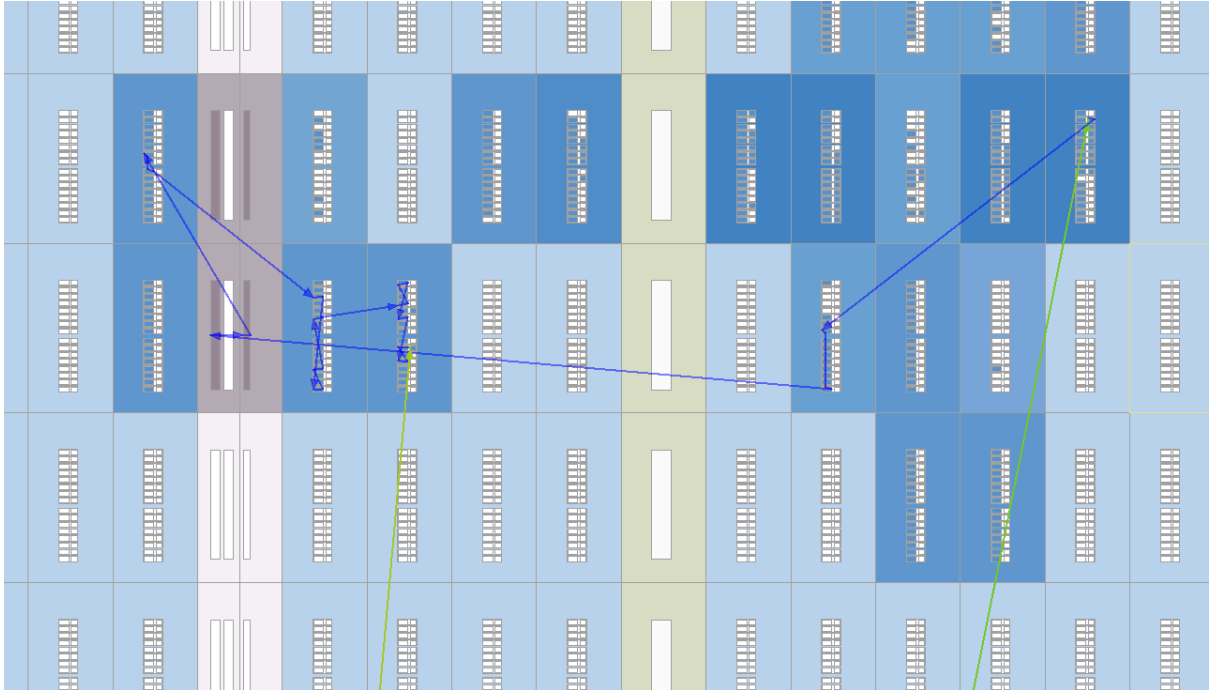
Si intuisce quindi che il path critico parta dal modulo **VGA\_sync** e arrivi al modulo **radar\_VGA**, attraversando una certa rete combinatoria, schematizzata in figura.

## 7.7 Path in chip planner

Si riporta il percorso critico all'interno del **Chip Planner**:



Questa rappresenta la disposizione fisica dei blocchi logici sul chip. I collegamenti blu tracciano il flusso del segnale tra i vari componenti. Si osserva come il path critico attraversi le diverse sezioni del chip nella maniera riportata in figura. Viene inoltre riportata la stessa immagine con l'aggiunta dell'arrival clock e del required clock (freccie verdi in figura):



## **8. Conclusioni**

E' dunque stato possibile realizzare una macchina digitale totalmente sincrona che permetta di implementare un semplice radar. La frequenza di clock utilizzata dal sistema è pari a 25 MHz. Mediante questa, sono stati ottenuti degli slack di setup e di hold positivi, che garantiscono che non si abbiano delle violazioni di tipo timing. Inoltre è stata gestita la criticità dei segnali asincroni in ingresso al sistema mediante delle operazioni di sincronizzazione, utilizzando dei semplici flip flop.

Inizialmente, era stato previsto l'utilizzo di due domini di clock: uno a 25 MHz per pilotare i moduli che utilizzano la VGA, e uno a 50 MHz per i restanti moduli. Sebbene, seguendo le regole del progetto sincrono statico, questo approccio fosse stato fattibile, è stata adottata una soluzione alternativa, come riportato nella relazione. Questa scelta ha permesso di evitare a priori la gestione di potenziali problemi di sincronizzazione tra moduli pilotati da domini di clock differenti.

La gestione della ROM è stata strutturata per coprire l'intero intervallo da 0° a 180°, nonostante solo la metà degli angoli fosse effettivamente utile, poiché il modulo del servomotore incrementa l'angolazione di 2° per volta. Questa scelta permette di adattare il progetto a diverse esigenze, ad esempio consentendo al radar di scandire un numero maggiore o minore di angoli semplicemente modificando l'incremento dell'angolo nel modulo del servomotore. Inoltre, riempire la memoria in questo modo ha reso possibile utilizzare direttamente la variabile dell'angolo in uscita dal modulo servo come indirizzo della ROM, evitando così operazioni di conversione.